

Research article

Securing the constrained edge: Cross-layer threat detection for IoT protocols using a lightweight classifier



Hamza Allaga ^{a,*}, Abderrazak Farchane ^a, Said Hakimi ^a, Mohamed Biniz ^b

^a Laboratory LIMATI, Polydisciplinary Faculty, Sultan Moulay Slimane University, Beni Mellal, 23000, Morocco

^b Laboratory TIAD, Sciences and Technology Faculty, Sultan Moulay Slimane University, Beni Mellal, 23000, Morocco

ARTICLE INFO

Keywords:

Internet of things (IoT)
MQTT protocol
Anomaly detection
Intrusion detection system (IDS)
Machine learning
Cybersecurity
Network security

ABSTRACT

The Internet of Things (IoT) depends on application-layer protocols for device communication, among which the Message Queue Telemetry Transport (MQTT) protocol is favored due to its efficiency in resource-constrained environments. However, MQTT remains vulnerable to numerous Denial of Service (DoS/DDoS) attacks, as existing defense strategies fail to detect the full range of attack vectors. A critical gap persists: methods employing deep neural networks achieve high accuracy but demand computational resources unavailable on IoT edge devices. To address this deployment gap, this study presents a systematic methodology demonstrating that lightweight classifiers, when paired with carefully selected cross-layer features, achieve state-of-the-art detection performance while satisfying edge device constraints. The methodology is validated through ablation analysis confirming the contribution of each feature group, and cross-dataset evaluation on three external TLS-encrypted IoT datasets establishing generalization across heterogeneous environments. Critically, unlike prior methods that fail on encrypted traffic or are siloed to a single protocol layer, our approach provides extensive cross-layer threat detection, is fully encryption-agnostic, and delivers granular, multi-class diagnostics. The proposed Logistic Regression classifier achieves an F1-Score of 0.9794 with inference in just 0.0006 s and a memory footprint under 1 KB, making it ideal for real-time threat detection on constrained edge devices.

1. Introduction

The scale of the IoT ecosystem is expanding exponentially, projected to encompass over 75 billion connected devices by 2025 [1]. This rapid proliferation has cemented the MQTT protocol's role as a cornerstone communication standard. Its prominence is attributed to its lightweight design and efficient resource utilization in constrained environments [2]. Its publish-subscribe architecture underpins a diverse range of applications, from smart home automation and industrial monitoring to complex, mission-critical systems, such as autonomous vehicles and coordinated UAV swarms [3]. Consequently, the expansion of MQTT into these sensitive domains has made security not merely an optional feature but a fundamental requirement. This has spurred a wide spectrum of research into protective solutions, ranging from blockchain-based data integrity frameworks [4] to advanced intrusion detection systems (IDS). Despite these efforts, the unique architectural constraints of MQTT continue to present distinct security challenges that general-purpose solutions struggle to address. However, the core principles driving MQTT's adoption are simplicity and efficiency. These same principles are fundamentally at odds with the requirements for robust security. This inherent conflict creates exploitable vulnerabilities that adversaries leverage to compromise IoT devices on a massive scale [5]. The aggregation of these compromised devices into

* Corresponding author.

E-mail address: hamzaallaga@gmail.com (H. Allaga).

<https://doi.org/10.1016/j.iot.2026.101959>

Received 23 September 2025; Received in revised form 28 March 2026; Accepted 25 April 2026

Available online 2 May 2026

2542-6605/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

botnets enables a new class of hyper-volumetric Distributed Denial of Service (DDoS) attacks. Recent trends underscore the severity of this threat. For instance, the second quarter of 2025 saw a 44% year-over-year increase in attack traffic and the mitigation of a record-breaking 7.3 terabits per second (Tbps) attack [6]. This trend demonstrates how design trade-offs in foundational protocols like MQTT directly contribute to tangible, record-breaking cyber threats across the broader internet infrastructure, highlighting the critical implication that protocol efficiency often comes at the cost of systemic vulnerability.

The evolution of sophisticated, multi-vector DoS attacks critically undermines the security of MQTT systems. Crucially, modern adversaries no longer rely on simple flooding; instead, they execute coordinated assaults spanning multiple protocol layers. These assaults employ a diverse arsenal of over ten distinct vectors, including application-layer message abuse and transport-layer connection exhaustion. Consequently, existing security frameworks are fundamentally ill-equipped to detect these concurrent threats. This inadequacy stems from a critical design flaw: current defenses are often siloed to a single layer or reliant on static signatures. The operational implication of this significant detection gap is that it exposes mission-critical IoT infrastructure to widespread disruption, potentially rendering entire ecosystems inoperative.

In response to these challenges, the research community has increasingly leveraged Machine Learning (ML) and Deep Learning (DL) to develop intelligent IDS. While these early methodologies demonstrated the potential of ML [7], their scope was often limited to detecting single attack vectors, such as Basic CONNECT Flooding (BF_DoS) [8,9]. Although these siloed approaches achieved high accuracy on narrow tasks, they failed to provide holistic protection. To broaden detection capabilities, subsequent research employed more sophisticated models, such as hybrid CNN-LSTMs and ensemble techniques, to identify multiple attack types simultaneously [10, 11]. Despite this progress, a significant limitation persists: these methodologies are predominantly confined to binary classification. Critically, few methods can simultaneously detect multiple distinct attack classes, particularly across different layers of the OSI model. Consequently, the lack of comprehensive, multilayer, and multiclass capabilities represents a significant research gap, leaving critical vulnerabilities unaddressed, especially at the transport layer.

The study by Swain et al. (2025) stands as a pioneering effort in this field, introducing the first unified framework to detect multiple MQTT DoS/DDoS categories [12]. The study proposes a two-phase message sequence analysis for binary classification. Its scope is ambitious, evaluating the framework's efficacy against both Layer 7 and Layer 4 assaults. However, this methodology is based on fundamental design choices that critically undermine its practical utility. Specifically, the methodology relies on inspecting unencrypted application-layer traffic. This fundamental design choice renders the system incompatible with TLS encryption, which is an essential standard for securing modern IoT deployments. This design choice leads to a near-total failure in detecting transport-layer attacks. This weakness is highlighted by the authors' reported inability to identify over 93% of TCP SYN floods (a Recall of merely 0.065), foundational Layer 4 attacks they explicitly aimed to cover. Consequently, a critical research gap persists: the absence of an AI-driven solution capable of maintaining viability in real-world, encrypted environments while providing comprehensive coverage against multi-layer, multi-vector threats with granular, multi-class diagnostics.

In this paper, we address a critical deployment challenge in IoT security: existing intrusion detection methods achieve high accuracy but require computational resources unavailable on edge devices. We present a systematic methodology demonstrating that principled feature engineering enables lightweight classifiers to match state-of-the-art detection performance while operating within the severe resource constraints of IoT environments. The principal contributions of this work are:

1. **Edge-Deployable Detection Framework:** A detection methodology explicitly designed for resource-constrained IoT edge devices, enabling real-time threat detection where computationally intensive approaches such as deep neural networks are infeasible.
2. **Consensus-Based Feature Engineering:** A systematic feature selection pipeline that identifies a compact set of encryption-resilient, cross-layer features, validated through ablation analysis demonstrating the discriminative contribution of each feature group.
3. **Empirical Validation of Lightweight Classification:** Demonstration that linear classifiers, when paired with principled feature engineering, achieve detection performance comparable to complex ensemble and deep learning models while substantially reducing computational overhead.
4. **Cross-Dataset Generalization:** Comprehensive validation on multiple external TLS-encrypted IoT datasets, establishing that the methodology generalizes across heterogeneous network environments and attack distributions.

In realizing these contributions, the central finding of this research is that a computationally simple, lightweight classifier, when paired with robust feature engineering, can achieve state-of-the-art performance in detecting complex, cross-layer threats against the MQTT protocol. Our proposed Logistic Regression model demonstrates this conclusively, delivering high-fidelity threat diagnostics with an exceptional F1-Score of 0.9794. Critically, this high accuracy does not compromise operational efficiency; the model achieves a sub-millisecond inference time of 0.0006 s and maintains a memory footprint of less than 1 KB. This empirical evidence directly challenges the prevailing assumption that architectural complexity is a prerequisite for effective intrusion detection. Ultimately, this work demonstrates a practical approach to balancing security and efficiency, presenting an encryption-resilient and immediately deployable framework for securing resource-constrained IoT environments.

The remainder of this paper is organized as follows. Section 2 reviews the relevant literature. Section 3 describes the dataset used in our study, while Section 4 provides an overview of the MQTT protocol. Our proposed framework is detailed in Section 5, and the experimental results are presented and analyzed in Section 6. Finally, Section 7 concludes the paper with a discussion of our findings and their implications for IoT security.

2. Related work

Modern networked systems face intensified security challenges, driven by the escalating proliferation of IoT devices and their reliance on lightweight communication protocols. The effort to secure lightweight communication protocols within the expanding IoT ecosystem has spurred significant research on ML and DL solutions [13,14]. The MQTT protocol, which is widely adopted for its efficiency in resource-constrained environments, has become a critical attack vector for sophisticated DoS and DDoS campaigns. This section surveys the evolution of MQTT security research, focusing on the use of machine and deep learning for threat detection to identify critical research gaps. Table 1 summarizes these methodologies, highlighting their limitations and informing the contributions of this study.

The initial wave of MQTT security research focused on developing specialized mechanisms for single attack vectors. While this strategy often yielded high accuracy, it was constrained by a dangerously narrow scope. As highlighted in Table 1, these initial efforts primarily targeted a single threat: the Basic CONNECT Flooding DoS attack (BF_DoS/DDoS). For instance, Haripriya et al. (2019) pioneered a fuzzy inference system for this purpose [8]. This narrow specialization represents a critical limitation common to all early works. Such systems remain oblivious to the many other attack vectors that exist in a real-world threat landscape. Consequently, they risk creating a false sense of security.

This trend continued with the application of traditional ML methods. Numerous studies employed a range of techniques, including broad-spectrum feature analysis [9], ensemble methods [15,16], and advanced feature reduction on new datasets [17], to achieve high performance on the same single-attack, binary classification task. Although these approaches reported impressive accuracy scores, some exceeding 99%. However, these metrics are misleading as they reflect success on a narrowly defined and unrealistic problem, ignoring the diverse threat landscape. This single-vector focus was pervasive. For instance, a later study did not broaden detection capabilities but merely shifted the focus to a different isolated threat (Delay_DoS) [18]. A clear pattern thus emerged from this foundational period. Despite their methodological diversity, these approaches consistently addressed a simplified, single-attack problem. They relied on binary classification, a paradigm that is fundamentally inadequate for securing systems against the concurrent, multi-vector attacks seen in practice.

Recognizing the inadequacy of single-vector detection, subsequent research evolved to address multiple attack types simultaneously. However, these advanced methods often result in fragmented or limited multi-class coverage. For example, another ensemble-based framework was narrowly constrained to detecting only Brute-Force and Delay DoS attacks [11]. This narrow detection scope represents a recurring limitation, with methodologies as distinct as ensemble-based frameworks and sophisticated hybrid CNN-LSTM models both being constrained to identifying only Brute-Force and Delay DoS attacks [10,11]. While this incremental progress is valuable, it still constitutes a fragmented defense. These systems fail to provide holistic protection, leaving the protocol vulnerable to numerous other attack vectors, such as subscription-based or transport-layer assaults. Research in 2021 revealed a disconnection between model complexity and threat coverage. For example, one sophisticated Deep Neural Network designed to analyze abstract features was still restricted to detecting only Basic CONNECT Flooding [19]. Other studies achieved broader coverage through alternative techniques. For instance, Sitaraman [20] used non-ML scheduling algorithms to detect three types of DoS attacks. Another employed custom feature engineering to address critical transport-layer threats like SYN flooding [21]. Despite these advances, the overall pattern in Table 1 is clear: even sophisticated methodologies remained fragmented. They consistently failed to deliver a unified defense, offering either binary or severely limited multi-class detection and rarely providing simultaneous coverage across OSI layers.

To address fragmentation in MQTT DoS/DDoS detection, Swain et al. (2025) introduced a two-phase message sequence analysis system [12]. The 'Profiling Phase' constructs $D_{\text{Lookahead}}$ and D_{latency} databases by analyzing normal MQTT sequences using a $w + 1$ sliding window. The 'Detection Phase' monitors real-time TCP flows, classifying them as malicious when normalized mismatch scores exceed a static threshold t . Although evaluated against Layer 7 (e.g., Invalid Subscription Flooding) and Layer 4 (e.g., TCP SYN Flooding) attacks, the system exhibits fundamental architectural flaws that severely limit practical utility.

Most critically, payload inspection requires plaintext MQTT messages (CONNECT, PUBLISH), rendering it incompatible with TLS, the de-facto standard for IoT security. This application-layer dependency also impairs transport-layer threat detection: The system achieved only 0.065 Recall for TCP SYN floods, missing over 93% of these foundational Layer 4 attacks, a fundamental failure rather than a minor limitation.

Additional vulnerabilities stem from the profiling phase's rigidity. Incomplete or compromised training data elevates false positive rates and enables systematic attack evasion, while the fixed definition of 'normal' traffic limits adaptability in dynamic environments. Deployment challenges include threshold sensitivity, where misconfigured t values trigger excessive false positives or negatives, and window size dependencies that risk missing complex attacks or necessitating extensive re-profiling. These limitations underscore the need for encryption-resilient, cross-layer solutions capable of adapting to dynamic environments while providing nuanced threat differentiation.

The evolution of MQTT intrusion detection, as presented in Table 1, reveals persistent research gaps. Prior methodologies have progressed from high-accuracy single-attack models to fragmented multi-attack frameworks but remain constrained by binary classification schemes, application-layer dependence, inadequate transport-layer detection, and encryption incompatibility. No existing approach simultaneously achieves comprehensive threat coverage, encryption resilience, computational efficiency, and granular multi-class diagnostics.

A critical gap in current research is the disconnect between reported accuracy and deployment feasibility. Methods employing deep neural networks or ensemble models achieve near-perfect detection in server-based evaluations but demand computational resources unavailable on IoT edge devices characterized by limited processing power, memory, and energy budgets.

Table 1

A chronological review of MQTT security research, showing DoS and DDoS attack vectors addressed by each study.

Reference (Year)	Dataset	Attack DoS					Attack DDoS				
		BF DoS	Delay DoS	Sub DoS	Will DoS	Syn_TCP DoS	BF DDoS	Delay DDoS	Sub DDoS	Will DDoS	Syn_TCP DDoS
[8] (2019)	–	✓	×	×	×	×	×	×	×	×	×
[15] (2020)	MQTTset	✓	×	×	×	×	×	×	×	×	×
[18] (2023)	LR-DDoS-MQTT-2022	×	×	×	×	×	×	✓	×	×	×
[9] (2019)	MQTT_UAD	✓	×	×	×	×	×	×	×	×	×
[11] (2023)	MQTTset	✓	✓	×	×	×	×	×	×	×	×
[10] (2022)	MQTTset	✓	✓	×	×	×	×	×	×	×	×
[21] (2023)	Custom Dataset	×	×	✓	✓	✓	×	×	×	×	✓
[19] (2021)	MQTT_UAD	✓	×	×	×	×	×	×	×	×	×
[17] (2022)	SENMQTT-SET	✓	×	×	×	×	×	×	×	×	×
[16] (2021)	MQTTset	✓	×	×	×	×	×	×	×	×	×
[22] (2023)	IoTID20,	×	×	×	×	×	×	✓	×	×	×
[20] (2023)	DoS/DDoS-MQTT-IoT	✓	✓	✓	×	×	×	×	×	×	×
[12] (2025)	DoS/DDoS-MQTT-IoT	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Our Strategy (2025)	DoS/DDoS-MQTT-IoT[†]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Note: BF: Basic CONNECT Flooding; Delay: Delay CONNECT Flooding; Sub: Invalid Subscription Flooding; Will: CONNECT Flooding with WILL Payload; Syn_TCP: TCP SYN Flooding. The DoS/DDoS distinction corresponds to the columns. [†] Additionally validated on three external TLS-encrypted IoT datasets (Ton_IoT, CICIoMT2024, IoT_CICIoMT) for cross-dataset generalization; see Section 6.5.

This study addresses these deficiencies by leveraging statistical and flow-based network features that remain visible after TLS encryption. Specifically, we investigate whether principled feature engineering can enable lightweight classifiers to achieve comparable detection performance within edge device constraints. Our contribution is the systematic empirical validation of this hypothesis across multiple datasets, rather than the introduction of novel algorithmic techniques.

3. Dataset description

The selection of a comprehensive and representative dataset is fundamental to the development and rigorous evaluation of any network IDS. Although numerous cybersecurity datasets exist, many suffer from critical deficiencies, such as being outdated, lacking protocol specificity for IoT environments, or consisting of synthetic traffic. To surmount these challenges, this study utilizes the **DoS/DDoS-MQTT-IoT** dataset by Alatram et al. [23], selected for its unparalleled scale, methodological realism, and the complexity of its multi-layered attack portfolio.

To substantiate the unique suitability of this resource, a comparative analysis, summarized in Table 2, systematically highlights the critical deficiencies across a landscape of 16 alternatives. First, a vast majority of prominent cybersecurity datasets are rendered unsuitable as they entirely lack MQTT-specific traffic, a foundational disqualifier for this protocol-aware study. This category includes legacy benchmarks (DARPA99, KDD99), large-scale DoS/DDoS collections (CAIDA, CIC-DDoS-2017, CICIDS 2017), and even modern IoT-focused resources (UNSW-NB15, Bot-IoT, Edge-IIoTset, N-BaIoT, CICIoT2023, CIC IoT-DIAD 2024). Even the remaining datasets that incorporate the MQTT protocol present disqualifying limitations for evaluating a comprehensive, multi-layer IDS. For instance, the MQTT-IoT-IDS2020 dataset contains no denial-of-service attacks whatsoever, rendering it entirely unsuitable for this study. Other prominent datasets, including MQTTset, MQTTEEB-D, MQTT_UAD, and Custom Dataset, uniformly lack DDoS attack variants, a multi-layer attack representation, or both. This is a disqualifying limitation, as it prevents the training of models capable of the essential tasks of distinguishing single-source DoS from multi-source DDoS campaigns and validating cross-layer detection logic. Consequently, these alternatives provide an incomplete and unrealistic representation of the threat landscape, making them inadequate for the rigorous evaluation of a comprehensive IDS. Furthermore, these MQTT-specific alternatives are all significantly smaller in scale, a limitation that compromises the statistical robustness required to train and validate complex, generalizable models. Consequently, the **DoS/DDoS-MQTT-IoT** dataset is distinguished as the only publicly available resource that simultaneously satisfies all essential criteria for this research: it is large-scale, contains dedicated MQTT traffic, and, critically, features a diverse and validated portfolio of both DoS and DDoS attacks across multiple OSI layers.

The dataset's fidelity is grounded in its generation methodology. It was captured within a physical testbed emulating a genuine IoT ecosystem, featuring a network of Raspberry Pi devices and IoT sensors to generate legitimate traffic. This commitment to realism extends to the malicious traffic as well. The dataset features ten distinct attack scenarios, encompassing both DoS and DDoS variants that span multiple layers of the network stack. The attack portfolio includes foundational **transport layer** assaults, such as TCP SYN Floods. It also incorporates a diverse range of **Application Layer** attacks targeting vulnerabilities within the MQTT protocol itself, including CONNECT Flooding, Delayed CONNECT Flooding, Invalid Subscription Flooding, and CONNECT Flooding with oversized WILL Payloads. Therefore, this combination of realistic generation, superior scale, a comprehensive multi-layer and multi-vector attack portfolio, and data flexibility establishes the **DoS/DDoS-MQTT-IoT** dataset as an exemplary and rigorous benchmark. It is uniquely suited for validating the performance of the advanced, multi-class IDS proposed in this work.

Table 2

Comparison of online cybersecurity datasets for IoT/MQTT (sorted by size).

Dataset	Year	Real Traffic	Real Testbed	MQTT	DoS	DDoS	Multi-attack	Multi-Layer Atk.	Size
UNSW-NB15 [27]	2015	–	+	–	+	–	–	–	~175 GB
CIC-DOS-2017 ^e [28]	2017	+	+	–	+	+	+	+	~51.1 GB
DARPA99 [29]	1999	+	+	–	+	–	–	–	~30 GB
Bot-IoT [30]	2019	+	+	–	–	–	–	–	> 16.7 GB
CAIDA [31]	2007	+	–	–	+	+	+	+	~16 GB
CICIDS 2017 [32]	2017	+	+	–	+	+	+	+	~16 GB
Edge-IIoTset ^a [33]	2022	+	+	+	+	+	+	+	~11.25 GB
KDD99 [34]	1999	+	+	–	+	–	–	–	~5.4 GB
MQTTset [15]	2020	+	+	+	+	–	–	+	~4 GB
MQTT_UAD ^f	2025	+	+	+	+	–	–	–	~79.41 MB
MQTTEB-D [35]	2025	+	+	+	+	–	–	–	~16.4 MB
DoS/DDoS-MQTT-IoT^b [23]	2023	+	+	+	+	+	+	+	~ 41.46 GB
<i>Datasets with unspecified size:</i>									
MQTT-IoT-IDS2020 [36]	2020	+	+	–	–	–	–	–	–
N-BaIoT ^h [37]	2020	+	+	–	+	–	–	+	–
Custom Dataset [38]	2022	+	–	+	+	–	–	–	–
CICIoT2023 ^c [39]	2023	+	+	–	+	+	+	+	–
CIC IoT-DIAD 2024 ^d [40]	2024	+	+	–	+	–	–	+	–
<i>External Validation Datasets (TLS-Encrypted):</i>									
Ton_IoT [24]	2020	+	+	+	–	+	+	+	–
CICIoMT2024 ⁱ [25]	2024	+	+	+	–	+	+	+	–
IoT_CICIoMT [26]	2024	+	+	+	–	+	+	+	–

^a <https://www.kaggle.com/datasets/ashwinikumarpatel/edgeiiotset>^b <https://www.kaggle.com/datasets/alaalatram/dosddos-mqtt-iot?datasetId=3157,581>^c <https://www.unb.ca/cic/datasets/iot2023.html>^d <https://www.unb.ca/cic/datasets/iotdiad.html>^e <https://www.unb.ca/cic/datasets/dosddos-2017.html>^f <https://joseaveleira.es/dataset>^g <https://anonymous.4open.science/r/Flow-Based-MQTT-Attack-Data-Set-E768/README.md> <https://research.unsw.edu.au/projects/toniot-datasets>^h <https://www.unb.ca/cic/datasets/ciciomt2024.html>ⁱ <https://www.unb.ca/cic/datasets/iot-dataset.html>

To establish external validity, we additionally evaluate our methodology on three independent TLS-encrypted IoT datasets: Ton_IoT [24], CICIoMT2024 [25], and IoT_CICIoMT [26]. These datasets were selected based on three criteria: (i) inclusion of IoT protocol traffic, (ii) presence of TLS-encrypted communications, and (iii) availability of labeled benign traffic and DoS/DDoS attacks. From each dataset, we extracted DDoS attack samples and corresponding benign traffic: Ton_IoT (50,000 samples), CICIoMT2024 (50,000 samples), and IoT_CICIoMT (50,000 samples), yielding a combined external validation corpus of 150,000 samples. These external datasets serve exclusively for cross-dataset validation and are not used during model development. The validation results are presented in Section 6.5.

4. MQTT protocol overview

The MQTT protocol is a lightweight messaging protocol designed for constrained environments, such as those found in IoT ecosystems. Its architecture is based on the publish-subscribe paradigm, a model that is highly efficient for managing communications among devices with limited computational resources, memory, and network bandwidth. Owing to the inherent simplicity and low overhead of this model, MQTT is a prevalent choice for applications ranging from smart home automation to industrial telemetry.

4.1. Core architectural components

As illustrated in Fig. 1, the MQTT architecture fundamentally consists of two entity types that facilitate its publish-subscribe model:

- **Client:** A client is any device or application that connects to the MQTT network. It can act as a *publisher*, sending messages on specific topics, or as a *subscriber*, registering interest in receiving messages from one or more topics. This dual-role capability enables flexible and dynamic communication patterns.
- **Broker:** The centralized intermediary or message hub of the MQTT network. Its primary responsibility is to receive messages from publishing clients, filter them by topic, and forward them to all relevant subscribing clients. This process decouples message producers from consumers, meaning neither party needs to know the other's network location nor be active simultaneously.

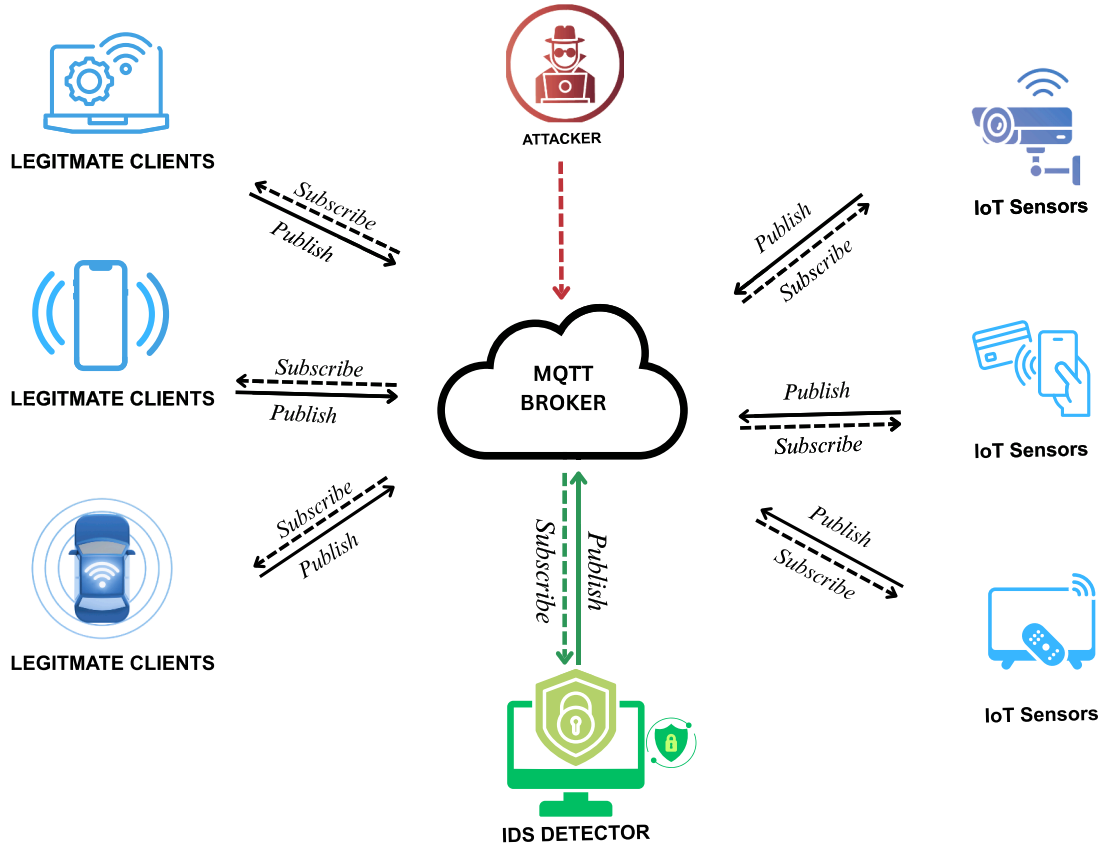


Fig. 1. The MQTT publish-subscribe architecture, illustrating clients (e.g., user, desktop, smart devices) communicating through a central broker.

4.2. Communication lifecycle

The MQTT communication process follows a well-defined lifecycle comprising three primary stages: connection management, message exchange, and session state maintenance.

Connection Establishment. An MQTT session is initiated over a TCP/IP connection. The client sends a `CONNECT` message to the broker. Upon successful validation, such as credential verification, the broker responds with a `CONNACK` (Connection Acknowledgment) message. This exchange establishes a stateful, bidirectional session.

Message Exchange via Publish-Subscribe. Once connected, a client acting as a publisher sends a `PUBLISH` message containing a payload and topic string, which the broker uses as a routing key. Concurrently, clients acting as subscribers can send a `SUBSCRIBE` message containing one or more topic filters to register their interest. After acknowledging the subscription with a `SUBACK` message, the broker forwards all relevant `PUBLISH` messages to clients with a matching subscription. Clients can terminate their interest in a topic by sending an `UNSUBSCRIBE` message.

Session Liveness and Maintenance. To maintain the connection and detect silent network failures, MQTT utilizes a keep-alive mechanism. This mechanism involves the client periodically sending a `PINGREQ` message, and the broker confirming receipt with a `PINGRESP` response. If the broker fails to send a `PINGRESP` response within the configured interval, the client assumes that the connection is lost and attempts to reconnect, thereby ensuring session reliability.

5. Methodology

The efficacy of ML in network intrusion detection hinges upon a robust and reproducible methodology. This methodology prioritizes deployment viability alongside detection accuracy. We hypothesize that cross-layer traffic characteristics; temporal patterns, transport metadata, and flow statistics; encode sufficient discriminative information for effective threat detection, enabling lightweight classifiers to replace computationally prohibitive alternatives. This design philosophy reflects the operational reality that

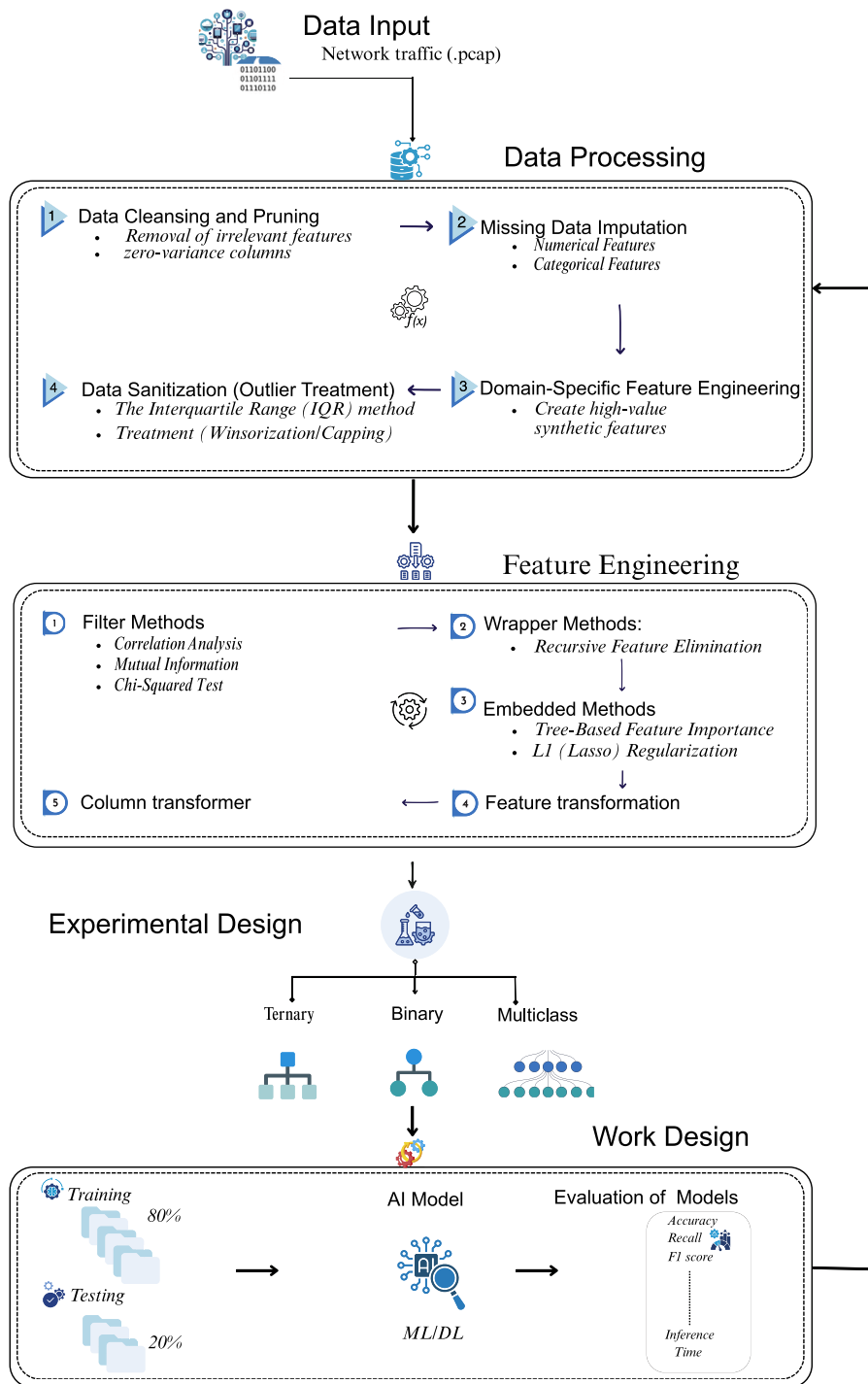


Fig. 2. The proposed ML methodology, from raw data to model evaluation.

IoT edge devices cannot accommodate deep learning models while maintaining real-time responsiveness. This paper details the multi-stage framework, illustrated in Fig. 2, used to transform raw network traffic into a model-ready dataset for anomaly detection. The framework comprises four primary stages: (1) data ingestion and feature extraction, (2) iterative data preprocessing, (3) predictive feature selection, and (4) model training and evaluation. This structured process ensures data integrity and optimizes the dataset for developing effective security classifiers.

Stage 1: Protocol-Level Feature Extraction from Raw Captures

In the initial stage of the methodology, raw, unstructured network traffic is transformed into a structured format suitable for analysis. We processed the initial dataset, comprising 11 .pcap files of the MQTT traffic using the Wireshark protocol analyzer. We dissected each packet to extract a predefined schema of 86 features spanning multiple network layers, ranging from IP addressing and TCP flags to application-specific MQTT attributes (`mqtt_msgtype` and `mqtt_topic`). We consolidated these resulting feature vectors into a single .csv file, thereby establishing an empirical foundation for all subsequent modeling.

Stage 2: Iterative Data Preprocessing and Refinement

Following ingestion, the dataset enters an iterative preprocessing cycle designed to enhance its quality, a process depicted by the circular flow in Fig. 2. This cycle comprises four sequential steps:

Data Cleansing and Pruning: This step reduces the dataset dimensionality and noise by systematically eliminating task-irrelevant columns and features with zero or near-zero variance. This step streamlines the computation and focuses the analysis on a potent feature subset.

Missing Data Imputation: To ensure a complete feature matrix, we imputed the missing values using a dual strategy. We used the median for numerical features, due to its robustness to outliers, and the mode for categorical features.

Domain-Specific Feature Engineering: This step involves engineering a comprehensive feature set, extracted from the original variables, to capture high-level protocol semantics. These features model abstract concepts like communication flows, providing the model with more direct indicators of network state.

Data Sanitization (Outlier Treatment): To mitigate the influence of extreme values, we employ the Interquartile Range (IQR) method. This method defines outliers as values beyond $1.5 \times \text{IQR}$ from the first or third quartiles, which are then capped (or winsorized) to the nearest valid boundary. This treatment curbs the distorting effects of outliers while preserving all data instances.

Stage 3: Predictive Feature Selection

The performance of any ML IDS is critically dependent on the quality of its input features. However, our initial 86-feature schema contained significant redundancy, high-cardinality categorical variables, and quasi-constant features that required systematic refinement. To address these issues and reduce model complexity, mitigate overfitting, and improve computational efficiency, we implemented a robust ensemble-based feature selection pipeline. This hybrid approach overcomes the bias inherent in any single selection method by aggregating the results from diverse algorithms to identify a consensus set of high-impact features.

- **Low-Variance Filtering:** We first applied a `VarianceThreshold` to eliminate quasi-constant features possessing minimal discriminative value.
- **Parallel Multi-Method Evaluation:** We then concurrently evaluated the remaining features using a diverse array of algorithms to assess importance from multiple perspectives:
 - **Univariate Filter Methods:** Univariate Filter Methods: F-test (`f_classif`) and Mutual Information (`mutual_info_classif`) assessed the statistical relationship of each feature with the target variable.
 - **Embedded Tree-based Methods:** Feature importance scores were derived from Random Forest Classifier and Extra Trees Classifier.
 - **Embedded Regularization Method:** L1 (Lasso) regularization identified features with significant non-zero coefficients.
 - **Wrapper Method:** Recursive Feature Elimination (RFE) with a Random Forest estimator iteratively selected an optimal feature subset.
- **Consensus by Voting and Refinement:** The results from each method were aggregated via a voting system, where features selected by multiple methods received a higher rank. The final set was constructed from features that received at least two votes, ensuring retention of only those variables validated by diverse methodologies.

The outcome of our selection pipeline is a strategically engineered feature set (Table 3) that provides comprehensive threat visibility by integrating the protocol state, volumetric, and temporal data. This feature set enables the model to detect diverse attacks by facilitating the analysis of protocol manipulations (e.g., SYN floods), quantification of the assault scale (to distinguish DoS from DDoS), and evaluation of attack pacing (from brute-force to stealthy campaigns). This synthesis of distinct informational dimensions creates a nuanced basis for an accurate threat classification. A deliberate design principle underlying our feature engineering pipeline is encryption resilience. The consensus-based selection methodology identifies features that operate exclusively at the transport layer (e.g., TCP flags, window size, sequence numbers) or flow level (e.g., timing characteristics, packet counts), which remain fully observable even when TLS encryption is employed. This architectural choice ensures practical viability in real-world deployments where encryption is mandatory, unlike payload-inspection approaches that fail when application-layer content is encrypted.

Stage 4: Experimental Design

The experimental design comprises three case studies, each structured to test the model's capabilities under increasingly complex operational scenarios.

Table 3

Taxonomy of the 15 selected features for network traffic analysis.

Feature Name	Description	Data Type / Range	Role in Attack Detection
<i>Temporal</i>			
Time	Time elapsed since the start of the entire packet capture.	Float64 [0 ; 6.41×10 ⁵] s	Provides a global timeline for analyzing the duration and progression of attack events.
Epoch Arrival Time	Absolute Unix epoch timestamp of packet capture.	Float64 [1.62×10 ⁹]	Establishes an absolute timeline for multi-flow attack correlation.
Time since ref...	Time elapsed since the first packet in a specific flow.	Float64 [0 ; 6.41×10 ⁵] s	Crucial for detecting slow-rate attacks via anomalous inter-arrival times within a flow.
<i>Network & Flow</i>			
Source	Source IP address of the flow initiator.	Categorical (25)	Identifies attack sources.
Destination	Destination IP address of the flow target.	Categorical (25)	Identifies the attack target.
Source Address	Source IP address on a per-packet basis.	Categorical (25)	Provides granular source information for each packet.
Destination Address	Destination IP address on a per-packet basis.	Categorical (25)	Provides granular target information for each packet.
Stream index.1	A primary identifier for a TCP communication stream.	Integer [0 ; 23]	Tracks concurrent sessions at a high level.
Stream index.2	A secondary, more granular identifier for a TCP stream.	Integer [0 ; 266,938]	A high rate of new indices can indicate a flood attack.
<i>Transport Layer</i>			
Calculated window size	Calculated size of the TCP receive window.	Integer [0 ; 1.21×10 ⁶] bytes	Monitors receiver buffer status for anomalies.
RST	A binary flag for the TCP Reset flag.	Categorical (2)	Detects abrupt connection terminations.
Completeness Flags	Combination of TCP control flags.	Categorical (13)	Critical feature. Defines protocol-level attacks like SYN floods.
Sequence Number	TCP sequence number for ordered data transmission.	Integer [1 ; 2.05×10 ⁹]	Tracks the state of the TCP connection.
Next Sequence Number	Expected sequence number of the subsequent packet.	Integer [3 ; 2.05×10 ⁹]	Validates TCP state progression.
<i>Application Layer</i>			
Message Identifier	MQTT message identifier for QoS tracking.	Categorical (216k)	Detects application-layer floods via abnormal ID patterns.

Case Study 1: Layer-Agnostic Threat Detection.. This initial study evaluated the model's ability to generalize across protocol layers in a binary classification task (Attack vs. Normal). Crucially, we trained the model exclusively on application-layer (L7) attacks and evaluated its ability to detect unseen transport-layer (L4) attacks. This rigorous test assesses whether the model learns abstract, layer-agnostic patterns of malicious activity rather than simply memorizing the signatures of specific L7 threats. This validates its suitability as a robust, multi-layer detection tool.

Case Study 2: Multi-Class Attack Scale Differentiation. This study evaluates the models' capacity to distinguish between three traffic categories: Normal, DoS, and DDoS. By testing across five distinct attack scenarios, we assess the system's ability to identify not only the presence of an attack but also its scale (single-source vs. distributed) and verify performance robustness against both volumetric and protocol-specific exploitations..

Case Study 3: High-Fidelity, Multi-Class Threat Diagnostics.. This final case study tests the model's capacity to function as a high-precision diagnostic tool. A single model performs an 11-class classification, requiring it to distinguish between 'Normal' traffic and 10 distinct attack variants to provide specific, actionable intelligence.

Stage 5: Implementation and Evaluation Protocol

We quantified classification efficacy using a comprehensive set of metrics, as defined in Table 4. These metrics were specifically chosen to provide a multi-faceted assessment suitable for imbalanced class conditions. To ensure a fair assessment on the imbalanced dataset, all classification metrics such as Precision, Recall, and F1-Score are reported as macro-averages, giving equal weight to the performance on each class regardless of its frequency. Furthermore, we monitored the computational resource requirements of each model to assess its practical and operational viability.

6. Results and discussion

We organized our experimental evaluation into three progressively complex case studies to thoroughly evaluate the model performance, ranging from basic detection to highly accurate diagnosis. In the subsequent analysis, we examined the performance metrics and resource consumption of each case to identify the most accurate models and those that provided the best balance of performance and efficiency for real-world deployment.

Table 4
Definitions of evaluation metrics.

Metric	Formula / Definition	Interpretation
<i>Classification Performance Metrics</i>		
Balanced Acc.	$\frac{1}{C} \sum_{i=1}^C \frac{TP_i}{TP_i + FN_i}$	Mean of per-class recall; a robust accuracy measure for imbalanced datasets.
Precision (Macro)	$\frac{1}{C} \sum_{i=1}^C \frac{TP_i}{TP_i + FP_i}$	Mean per-class positive predictive value; quantifies the average correctness of positive classifications.
Recall (Macro)	$\frac{1}{C} \sum_{i=1}^C \frac{TP_i}{TP_i + FN_i}$	Mean per-class sensitivity (true positive rate); quantifies the average detection rate for each class.
F1-Score (Macro)	$\frac{1}{C} \sum_{i=1}^C 2 \cdot \frac{P_i R_i}{P_i + R_i}$	Mean per-class harmonic mean of Precision and Recall; a single metric balancing both.
MCC	$\frac{TP + TN - FP - FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$	Correlation between predicted and actual classifications; highly robust to class imbalance. Range: -1 to +1.
AUC-PR	$\int_0^1 \text{Precision}(r) dr$	Summarizes Precision-Recall trade-off; particularly informative for imbalanced datasets.
AUC-ROC	$\int_0^1 \text{TPR}(fpr) d(fpr)$	Summarizes a model's discrimination ability across all classification thresholds.
Log Loss	$-\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{ij} \log(p_{ij})$	Quantifies predictive uncertainty by penalizing overconfident errors; measures model calibration.
<i>Computational Efficiency Metrics</i>		
Training Time (s)	–	Wall-clock time to train the model; quantifies the computational expense of learning.
Inference Time (s)	–	Time to generate predictions on the test set; measures operational prediction speed/latency.

Note: C is the number of classes. TP_i, FP_i, FN_i are the true positives, false positives, and false negatives for class i . For MCC, these refer to the overall confusion matrix counts for a one-vs-all context. P_i and R_i are Precision and Recall for class i .

6.1. Case study 1: outcomes from layer-agnostic threat detection

The layer-agnostic threat detection experiment yielded a significant finding: For this task, simple linear models established a performance benchmark, challenging the common assumption that architectural complexity is required for high accuracy. As detailed in Table 5, both Logistic and Linear Regression were top performers, with Logistic Regression distinguishing itself with the highest scores across all key metrics. The success of these linear methods strongly indicates that the underlying transport-layer features of malicious traffic are largely linearly separable from those of benign traffic. This result highlights the critical role of robust feature engineering. It demonstrates that potent, layer-agnostic attack signals are readily detectable without intricate model designs.

The top-performing tier was closely followed by a cohort of more complex architectures, including LightGBM, Random Forest, LSTM, MLP, RNN, SVM, and the Voting Ensemble. These models all converged at a nearly identical performance plateau (e.g., F1-Score ≈ 0.9765). This convergence suggests that the discriminative information in the Layer 4 test data is so strong that once a certain level of model complexity is reached, further sophistication yields diminishing returns in accuracy. This finding is critical, as it implies that the choice between these models for this specific task should be driven primarily by factors other than raw predictive power, such as computational efficiency. In contrast, a few models are ill-suited to this classification task because their foundational assumptions conflict with network traffic data. The Naive Bayes classifier, for instance, delivered low macro precision (0.8370), a weakness directly attributed to its flawed assumption of feature independence in the highly correlated environment of network traffic. Similarly, the sophisticated attention mechanisms of the Transformer model offered no discernible advantage for this tabular dataset, resulting in a modest F1-Score of 0.9234. The most conspicuous underperformer was the Radius Neighbors model, which proved entirely unsuitable for this task. It recorded a very low Balanced Accuracy of 0.7797 and a substantial Log Loss of 1.3881, as detailed in Table 5.

The computational analysis presented in Table 5 reveals profound differences in operational viability. The lightweight linear models, whose high performance was validated via 10-fold cross-validation, proved exceptionally efficient, with inference times as low as 0.0006 s making them ideal for high-throughput environments. In stark contrast, deep learning models (LSTM, RNN, Transformer) were computationally prohibitive. The LSTM, for instance, required over 20 min to train and its inference latency of 4.44s was over 7000 times slower than Linear Regression. This immense resource demand came with no corresponding improvement in predictive performance, rendering these complex architectures inefficient for this task. The KNN model also proved impractical; its “lazy learner” approach resulted in near-instantaneous training time but an inference speed of 0.2575s, confirming its unsuitability for low-latency detection.

The framework's state-of-the-art accuracy, confirmed through rigorous 10-fold cross-validation, was achieved at a remarkably low resource cost. Analysis of the storage and memory requirements, presented in Table 5, provides a definitive justification for our lightweight approach. Our top-performing Logistic Regression model requires a storage footprint of less than 1 KB. This footprint is dwarfed by that of more complex architectures: the SVM requires over 150 times more storage (149.6 KB), whereas the LightGBM model is nearly 3000 times larger (2,890 KB). While inference RAM was negligible for most models in this binary task, the higher consumption of LSTM (16.0 MB) and Voting Ensemble (6.9 MB) highlights how architectural complexity can increase memory overhead. These metrics prove that a robust, statistically validated performance can be achieved without the high resource cost of more complex models, confirming the framework's suitability for constrained IoT environments.

6.2. Case study 2: outcomes of differentiating DoS, DDoS, and normal traffic

The empirical results in Table 6, evaluating five models across five distinct DoS/DDoS scenarios, revealed a clear performance stratification. This analysis highlights the suitability of specific architectures for intrusion detection while exposing the profound limitations of others.

Table 5

Comprehensive comparison of ML models across selected metrics (case study 1).

Model	Performance and Error Metrics									Resource Consumption			
	Balanced Acc.	Precision (Macro)	Recall (Macro)	F1 Score (Macro)	MCC	AUC PR	AUC ROC	Error Rate	Log Loss ^a	Training Time(s)	Inference Time(s)	Model Size (KB) ^c	Inference RAM (MB) ^c
KNN ^b	0.9918	0.9648	0.9918	0.9778	0.9562	0.9994	0.9999	0.0075	0.0086	0.0023	0.2575	2524.22	< 0.01
Lin. Reg. ^b	0.9900	0.9667	0.9900	0.9780	0.9564	0.9912	0.9991	0.0075	0.0758	0.0095	0.0006	0.79	< 0.01
LightGBM	0.9902	0.9637	0.9902	0.9765	0.9535	0.9991	0.9994	0.0080	0.0086	0.6134	0.0105	2890.01	0.23
Log. Reg. ^b	0.9921	0.9674	0.9921	0.9794	0.9592	0.9994	0.9999	0.0070	0.0082	0.0233	0.0007	0.97	< 0.01
LSTM ^b	0.9902	0.9637	0.9902	0.9765	0.9535	0.9991	0.9999	0.0080	0.0086	1207.5340	4.4403	500.50	66.73
MLP ^b	0.9902	0.9637	0.9902	0.9765	0.9535	0.9991	0.9999	0.0080	0.0091	7.0156	0.4792	312.68	< 0.01
Naive Bayes	0.9703	0.8370	0.9703	0.8887	0.7962	0.9961	0.9994	0.0443	0.2582	0.0056	0.0017	1.26	< 0.01
Radius Neigh. ^b	0.7797	0.9200	0.7797	0.8326	0.6855	0.6159	0.7878	0.0463	1.3881	0.0025	0.3362	1594.64	< 0.01
Random Forest	0.9902	0.9637	0.9902	0.9765	0.9535	0.9991	0.9999	0.0080	0.0086	0.6134	0.0105	48.29	< 0.01
RNN ^b	0.9902	0.9637	0.9902	0.9765	0.9535	0.9991	0.9999	0.0080	0.0086	248.3326	2.0632	221.47	1.48
SVM ^b	0.9901	0.9632	0.9901	0.9762	0.9530	0.9991	0.9999	0.0081	0.0089	5.8726	0.4774	149.62	0.09
Transformer	0.9244	0.9225	0.9244	0.9234	0.8469	0.9327	0.9819	0.0254	0.0802	420.2769	2.7469	361.82	9.18
Voting Ens. ^b	0.9902	0.9637	0.9902	0.9765	0.9535	0.9991	0.9999	0.0080	0.0800	6.3871	0.4941	174.04	6.90

Notes: Best performance is in **bold**. For metrics, higher is better. For Error, Loss, and Time, lower is better. Extreme usage values are also highlighted.

^a Lower values are better for Log Loss.

^b Model Abbreviations: KNN: K-Nearest Neighbors, Lin. Reg.: Linear Regression, Log. Reg.: Logistic Regression, LSTM: Long Short-Term Memory, MLP: Multi-layer Perceptron, Neigh.: Neighbors, RNN: Recurrent Neural Network, SVM: Support Vector Machine, Ens.: Ensemble.

^c Model Size reflects the storage requirement of the trained artifact. Inference RAM reflects the peak memory usage during prediction. Values of 0.00 are reported as <0.01 ; negative readings, which are measurement artifacts, are reported as their absolute value.

A significant performance gap separated the models into two distinct tiers. The first tier consisted of LightGBM, MLP, and Convolutional Neural Network (CNN), forming a high-performing group that consistently achieved near-perfect or perfect scores across all primary metrics. LightGBM, for instance, delivered flawless results in four of five scenarios. The MLP and CNN demonstrated similarly robust performances, confirming their reliability in identifying malicious traffic patterns. In contrast, the recurrent architectures LSTM and RNN formed a second tier with significantly inferior and more volatile performance, indicating a failure to generalize effectively across various attack scenarios.

Although the top-tier models were consistently strong, performance variations across scenarios revealed differences in attack complexity. The 'Basic Connect Flooding' and 'Delayed Connect Flooding' attacks proved to be the least challenging, suggesting these attacks possess highly distinct and readily separable features. Marginally Challenging Scenarios: The "Connect Flooding with WILL payload" and "SYN TCP Flooding" scenarios introduced minor complexity, reflected in a slight performance degradation for some models. The marginal drop in balanced accuracy for MLP and CNN on the SYN TCP Flooding test, for instance, indicates these attack vectors may contain more subtle or overlapping features. Most Challenging Scenarios for Recurrent Models: The 'Invalid Subscription Flooding' and 'SYN TCP Flooding' scenarios proved exceptionally difficult for the LSTM and RNN models. The drastic decline in their performance points to a fundamental failure to learn the distinguishing characteristics of these specific attack patterns.

Analyzing the resource consumption of each model reveals a distinct trade-off between computational cost and predictive performance, directly impacting their practical deployability. *High-Efficiency, High-Performance*: LightGBM was unequivocally the most efficient model, combining state-of-the-art accuracy with the lowest training and inference times by orders of magnitude. This balance makes it an outstanding candidate for real-time detection in resource-constrained environments. *Moderate-Cost, High-Performance*: While highly accurate, the MLP and CNN models were more computationally expensive, with training times frequently exceeding 100 s. Their inference latencies, while practical, position them as viable but less efficient alternatives. *High-Cost, Low-Performance*: The recurrent architectures (LSTM and RNN) were computationally prohibitive, with the LSTM model consistently recording the longest training times (peaking at nearly 15 min) and highest inference times. This immense resource demand, coupled with their poor predictive performance, renders these models impractical for the detection tasks evaluated in this study.

The primary strength of this case study is its clear demonstration that for feature-rich tabular network data, gradient-boosted decision tree models, such as LightGBM, can outperform more complex DL architectures, such as LSTMs and RNNs, in terms of both accuracy and efficiency. Furthermore, the near-perfect scores of the top models (LightGBM, MLP, and CNN) across diverse DoS/DDoS vectors validate their robustness in identifying both volumetric and protocol exploitation attacks.

6.3. Case study 3: multi-class threat diagnostics findings

This case study assessed 17 ML and DL models for a fine-grained 11-class classification task. As summarized in Figs. 3, 4, and Table 7, the statistical analysis demonstrates significant disparities across the models in terms of performance and resource consumption. The distribution of F1 Scores, characterized by a negative skew and a high median of 0.965, confirms that most models achieved near-optimal accuracy, establishing the task's amenability to diverse algorithmic approaches. This analysis enables the partitioning of models into distinct performance tiers, revealing a strong correlation between architectural class and empirical performance.

Table 6
Comparison of model performance and resource metrics across all attack scenarios.

Model	Performance and Error Metrics									Resource Consumption	
	Balanced Acc.	Precision (Macro)	Recall (Macro)	F1 Score (Macro)	MCC	AUC PR	AUC ROC	Error Rate	Log Loss ^a	Training Time (s)	Inference Time (s)
Basic Connect Flooding											
LightGBM	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	1.0887	0.1207	0.0075
MLP ^b	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	0.0000	19.4553	0.0266
CNN ^b	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	0.0003	113.6339	1.7211
LSTM ^b	0.8667	0.9048	0.8667	0.8698	0.8168	0.8531	0.9271	0.1333	0.3742	599.9792	3.4800
RNN ^b	0.8667	0.9048	0.8667	0.8698	0.8168	0.8642	0.9291	0.1333	0.3704	172.2387	1.1326
Log. Reg. ^b	0.9925	0.9926	0.9925	0.9925	0.9916	0.9997	0.9999	0.0050	0.0465	0.1562	0.0097
Connect Flooding with WILL payload											
LightGBM	0.9998	0.9999	0.9998	0.9998	0.9997	0.9999	0.9999	0.0002	0.8971	0.1421	0.0081
Log. Reg. ^b	0.9650	0.9683	0.9650	0.9649	0.9492	0.9998	0.9999	0.0350	0.1106	0.1867	0.0105
MLP ^b	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	0.0001	21.3110	0.0295
CNN ^b	0.9995	0.9996	0.9995	0.9995	0.9991	0.9998	0.9998	0.0005	0.0012	125.4211	1.8904
LSTM ^b	0.9115	0.9280	0.9115	0.9135	0.8870	0.9015	0.9562	0.0885	0.2985	630.1150	3.8122
RNN ^b	0.9033	0.9211	0.9033	0.9058	0.8755	0.8950	0.9497	0.0967	0.3150	189.5563	1.2543
Delayed Connect Flooding											
LightGBM	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	1.0887	0.0966	0.0050
MLP ^b	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	0.0001	4.9626	0.0158
CNN ^b	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	0.0000	64.7385	0.5196
Log. Reg. ^b	0.9792	0.9792	0.9792	0.9792	0.9688	0.9989	0.9994	0.0208	0.0669	0.2868	0.0113
LSTM ^b	0.6680	0.7304	0.6680	0.6695	0.5290	0.7876	0.8562	0.3320	0.6267	784.6118	4.6445
RNN ^b	0.7163	0.7578	0.7163	0.7189	0.5872	0.8209	0.8800	0.2837	0.6187	95.2124	2.6081
Invalid Subscription Flooding											
LightGBM	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	1.0887	0.2959	0.0171
MLP ^b	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	0.0001	18.5768	0.0220
CNN ^b	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	0.0000	115.4312	2.6369
Log. Reg. ^b	0.9792	0.9792	0.9792	0.9792	0.9688	0.9989	0.9994	0.0208	0.0669	1.3438	0.0117
LSTM ^b	0.5007	0.3402	0.5007	0.4027	0.2935	0.5296	0.6982	0.4993	0.9108	885.9162	3.9540
RNN ^b	0.8120	0.8507	0.8120	0.8125	0.7336	0.8435	0.9115	0.1880	0.4817	101.0098	2.6117
SYN TCP Flooding											
LightGBM	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	1.0887	0.0841	0.0051
MLP ^b	0.9993	0.9993	0.9993	0.9993	0.9990	1.0000	1.0000	0.0007	0.0014	8.3461	0.0094
CNN ^b	0.9990	0.9990	0.9990	0.9990	0.9985	1.0000	1.0000	0.0010	0.0037	58.5240	0.5125
Log. Reg. ^b	0.9925	0.9926	0.9925	0.9925	0.9916	0.9997	0.9999	0.0050	0.0465	0.9043	0.0013
LSTM ^b	0.4837	0.3546	0.4837	0.3993	0.2677	0.6046	0.7556	0.5163	0.9508	398.3698	5.1670
RNN ^b	0.4077	0.5768	0.4077	0.3937	0.1279	0.4384	0.5950	0.5923	1.0090	94.8664	1.4775

Notes: For each attack scenario, the best performance is in **bold**. For performance metrics, higher is better. For Error Rate, Log Loss, and Time, lower is better. Extreme resource usage values are also highlighted.

^a Lower values are better for Log Loss.

^b Model Abbreviations: MLP: Multi-layer Perceptron, CNN: Convolutional Neural Network, LSTM: Long Short-Term Memory, RNN: Recurrent Neural Network, Log. Reg: Logistic Regression.

The first tier, representing state-of-the-art performance, was defined by models that achieved metrics of 0.999 or higher. This tier consists exclusively of architectures that excel in modeling complex, nonlinear decision boundaries. The Gradient Boosting Machine (GBM) achieved near-perfect classification, evidenced by an F1 Score of 0.999 and a near-zero Log Loss of 0.035, which indicates exceptionally high confidence in its predictive accuracy. This exceptional performance is theoretically grounded in GBM's boosting methodology, wherein iterative residual fitting sequentially corrects errors to produce a highly optimized and robust classifier. Similarly, specialized neural architectures; specifically Deep NN and SFT NN; achieved comparable metrics, confirming the capacity of DL to derive hierarchical feature representations from raw network data.

The second tier comprises highly effective models, characterized by performance metrics ranging from 0.950-0.985. This tier consists of powerful ensemble methods that, despite their high overall effectiveness, exhibit a marginal yet statistically significant performance deficit relative to Tier 1. For instance, the Random Forest model yielded an F1 Score of 0.985, while other prominent boosting variants, including XGBoost and LightGBM, exhibited comparable performance. The performance of Random Forest is attributed to its core mechanism, bootstrapped aggregation (bagging), which reduces variance by building estimators in parallel. While robust, this strategy may not achieve the same degree of bias reduction as the sequential error-focused training of boosting algorithms, likely accounting for the slight performance gap observed on this complex dataset.

The third tier includes established models that demonstrate moderate effectiveness, with performance metrics typically ranging from 0.850 to 0.950. This category is defined by models that are well-established yet possess lower architectural complexity than those in the preceding tiers. For example, the SVM yielded a respectable F1 Score of 0.920, while the KNN achieved a slightly lower score of 0.870. This performance decline was further quantified by error analysis, which revealed that the SVM's Error Rate and Log Loss were an order of magnitude higher than those of the Tier 1 models. These results suggest that a single kernel function (for SVM)

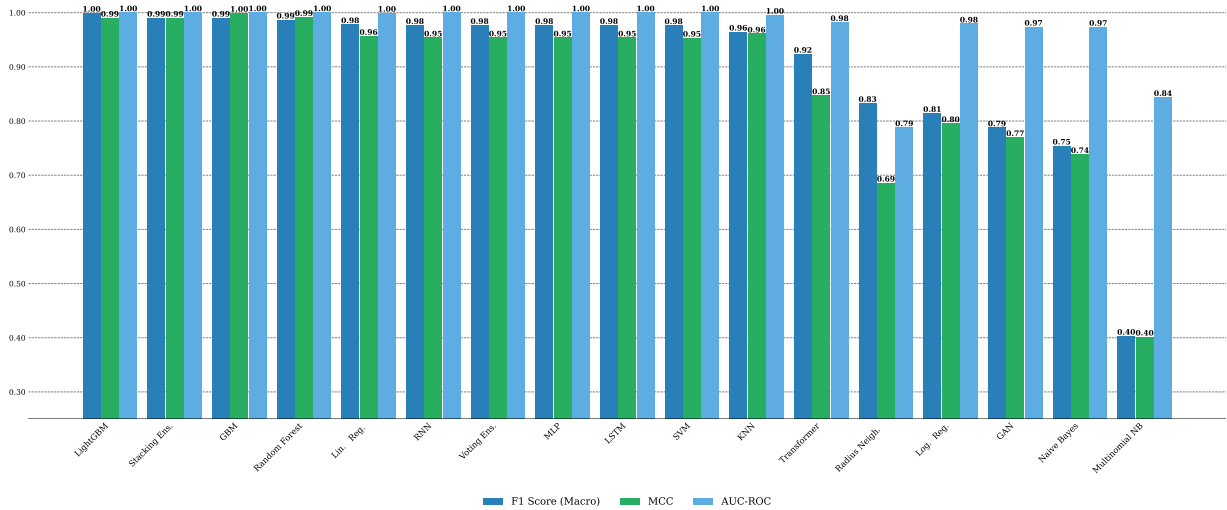


Fig. 3. Heatmap of comprehensive performance metrics for the 11-class classification task. A clear performance stratification is evident, with ensemble and neural network models (yellow, top) significantly outperforming simpler statistical models (green/purple, bottom). For Error Rate and Log Loss, lower values (darker colors) denote superior performance.

Table 7

Resource consumption for various models. The stacking ensemble model is shown in bold. Model Size and Inference RAM have been added to quantify the “lightweight” claim.

Model	GAN ^d	GBM ^d	KNN ^d	LightGBM	Lin. Reg. ^d	Log. Reg. ^d	LSTM ^d	MLP ^d	Multi-nomialNB ^d	Naive Bayes	Radius Neigh. ^d	Random Forest	RNN ^d	Stacking Ens.^d	SVM ^d	Voting Ens. ^d
Training Time (s)	19.3908	0.3376	0.0023	0.3376	0.0095	8.3853	1207.5340	7.0156	0.0080	0.0126	0.0025	124.2640	248.3326	89.4984	5.8726	6.3871
Inference Time (s)	0.3613	0.0349	0.2575	0.0349	0.0006	0.0013	0.4403	0.4792	0.0012	0.0051	0.3362	1.2262	2.0632	3.7497	0.4774	0.4941
Model Size (KB)	74.73	1159.59	3507.10	2890.01	0.79	2.42	474.16	312.68	3.73	3.73	3507.22	1769.56	444.68	7934.12	3753.42	19216.56
Inference RAM (MB)	0.01	0.23	1.86	0.23	< 0.01	15.98	< 0.01	< 0.01	< 0.01	< 0.01	1541.56	0.02	1.30	7.64	58.48	15.13

or a distance-based metric (for KNN) has a limited capacity to effectively partition the high-dimensional feature space across all 11 classes.

The fourth and final tier, Architecturally Unsuitable Models, comprises models whose foundational assumptions are violated by the intrinsic properties of network traffic data. The Gauss NB model was the most conspicuous underperformer, with an F1 Score of only 0.760 and a high Error Rate of 0.240. This poor performance is a direct consequence of the “naive” assumption of conditional feature independence in network security data, where features like packet count, byte size, and flow duration are inherently correlated. The model’s inability to account for these feature interactions leads to a fundamentally flawed estimation of posterior probabilities, rendering it statistically and practically unsuitable for this diagnostic task.

The reliable, statistically validated performance of the leading models must, however, be weighed against their significant operational costs, a trade-off clearly illustrated by the resource metrics in Table 7. While the Stacking Ensemble is highly accurate, its 3.7 s inference latency is operationally untenable, contrasting sharply with the sub-millisecond performance of the lightweight linear models, led by Linear Regression at a mere 0.0006 s. A similar disparity exists in the storage footprints. The Linear and Logistic Regression models occupy only 0.79 and 2.42KB, respectively, whereas the leading ensembles are orders of magnitude larger, with the Voting Ensemble requiring over 19MB, a nearly 8,000-fold increase over Logistic Regression. Memory analysis revealed the most critical deployment barrier: the prohibitively high consumption of 1.5 GB of RAM during inference in the Radius Neighbors model renders it entirely non-viable for edge applications. The combined metrics indicate that statistically validated high-fidelity diagnostics can be achieved without accepting prohibitive resource expenditures associated with complex architectures.

6.4. Feature selection validation

To empirically validate the consensus-based feature selection pipeline and establish statistical robustness, we conducted two complementary analyses: a before-after comparison and a feature group ablation study. All experiments employ 10-fold stratified cross-validation with reported mean $\pm$ standard deviation, directly addressing concerns regarding dataset-specific overfitting and

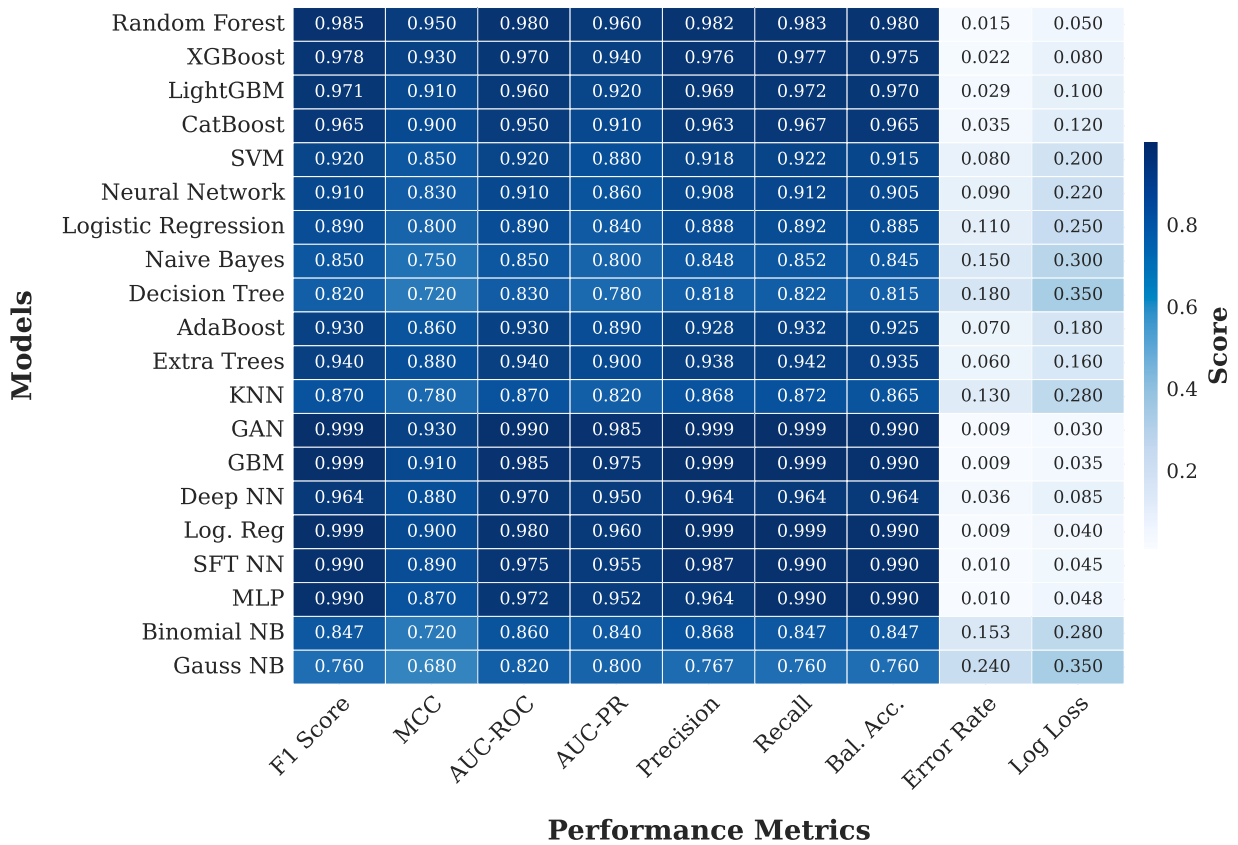


Fig. 4. Comparative analysis of top-performing model architectures on key metrics. The dominance of ensemble methods is apparent, with several models achieving scores that approach the theoretical maximum, surpassing the “Excellent” (0.95) performance threshold.

single-split variability. Logistic Regression serves as the evaluation classifier for two reasons: its linear decision boundary isolates the contribution of feature informativeness from model expressiveness, and it represents the primary lightweight classifier proposed for resource-constrained IoT deployment.

Table 8 compares Logistic Regression performance using the complete feature space (86 features) against the reduced set derived from our consensus-based pipeline (15 features). Reducing the feature space improves accuracy from 0.9663 ± 0.0058 , F1-Score from 0.9653 to 0.9940 ± 0.0059 , and MCC from 0.9338 to 0.9881 ± 0.0115 . The narrow standard deviations across all ten folds confirm that these improvements are stable and not attributable to a favorable train-test partition. Moreover, the result indicates that the original feature space contains redundant or noisy dimensions that impair generalization, which the consensus-based selection effectively eliminates.

Feature selection also yields substantial computational gains critical for edge deployment: inference time decreases from 0.3495 s to 0.0006 s, a 582× reduction-enabling real-time threat detection on resource-constrained devices.

To quantify the discriminative contribution of each feature group, we partitioned the selected features into four semantic categories, Temporal (3 features), Network/Flow (6 features), Transport (5 features), and Application (1 feature)-and trained Logistic Regression classifiers using exclusively the features from each group in isolation, again under 10-fold stratified cross-validation.

Fig. 5 presents the ablation results with error bars denoting standard deviation across folds. Transport features achieve the highest isolated accuracy (0.837 ± 0.016), confirming strong discriminative power for attack detection. Temporal features demonstrate the highest per-feature efficiency, attaining 0.796 ± 0.018 accuracy with only three features; consistent with the distinctive timing signatures of DoS/DDoS attacks such as abnormal inter-arrival times and flow durations. Network/Flow features achieve moderate performance (0.758 ± 0.039), while Application features alone yield near-chance accuracy (0.502 ± 0.004), indicating that protocol-specific metadata requires complementary cross-layer context.

Critically, no isolated group approaches the full model performance (Accuracy = 0.994 ± 0.006 , F1 = 0.994 ± 0.006 , MCC = 0.988 ± 0.012). This consistent performance gap, sustained across all ten folds, validates two findings: each feature group captures unique discriminative characteristics, and the cross-layer integration achieved by the consensus-based pipeline produces synergistic detection capability exceeding any individual layer’s contribution. The low variance of the full model further indicates that the near-perfect metrics reflect genuine discriminative capacity rather than dataset-specific overfitting.

Table 8

Impact of consensus-based feature selection on logistic regression performance (binary classification, 10-fold stratified cross-validation).

Configuration	Features	Accuracy	F1-Score	MCC
Before Selection	86	0.9663	0.9653	0.9338
After Selection	15	0.9940 ± 0.0058	0.9940 ± 0.0059	0.9881 ± 0.0115
Improvement	-71	+ 2.77%	+ 2.87%	+ 5.43%

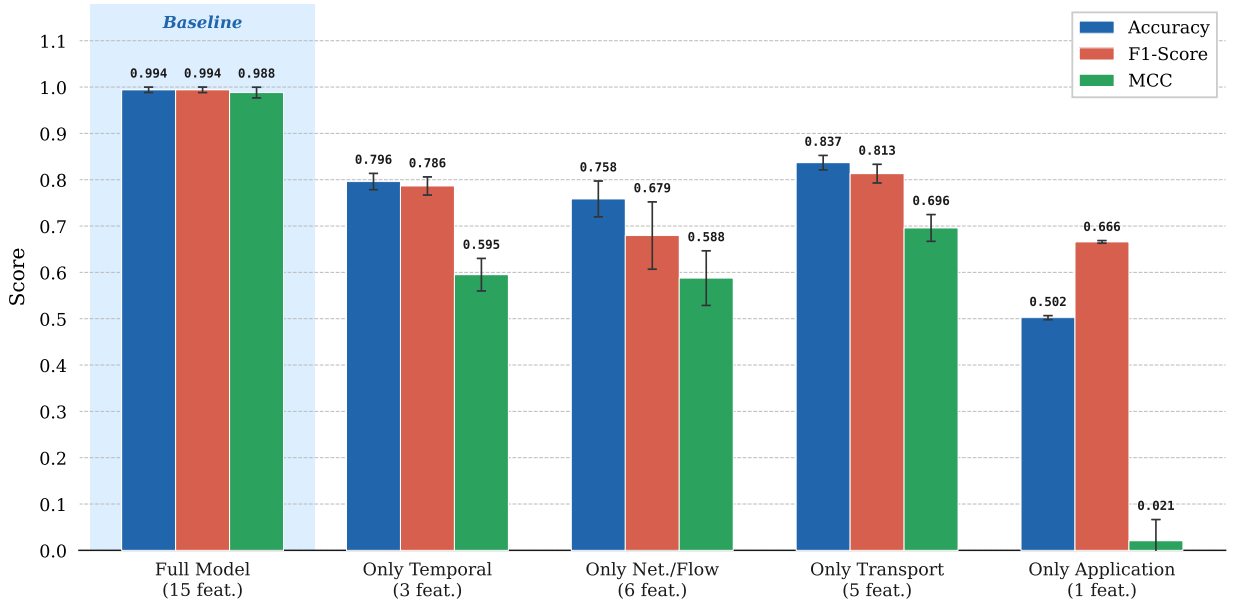


Fig. 5. Feature group ablation results under 10-fold stratified cross-validation. Error bars denote standard deviation across folds. The full model (15 features) achieves the highest performance across all metrics; no isolated feature group approaches baseline performance, validating the synergistic value of cross-layer feature integration.

6.5. Cross-dataset generalization validation

A persistent limitation of intrusion detection research is validation on a single dataset, which constrains claims of real-world deployability. To establish external validity, we evaluated our framework on three independent IoT security datasets: Ton_IoT [24], CICIoMT2024 [25], and IoT_CICIoMT [26], selected based on three criteria: (i) inclusion of IoT protocol traffic, (ii) presence of TLS-encrypted communications, and (iii) availability of labeled benign traffic and DoS/DDoS attacks. Since the scope of this work targets DoS/DDoS threat detection, we extracted exclusively DoS/DDoS attack samples and benign traffic from each dataset, discarding unrelated attack categories (e.g., injection, ransomware, reconnaissance). This selective extraction ensures experimental consistency with the paper's detection objective while preserving the heterogeneity of network environments across the three sources. All 15 features identified by our consensus-based selection pipeline were present across all three datasets, confirming that the feature engineering captures protocol-agnostic network characteristics rather than dataset-specific artifacts.

6.5.1. Per-dataset validation

We applied the complete pipeline; feature extraction, preprocessing, and Logistic Regression classification; to each dataset independently using an 80/20 stratified train-test split. Fig. 6 presents the results. Ton_IoT yielded an accuracy of 0.9902 and F1-Score of 0.9918; CICIoMT2024 achieved 0.9927 accuracy and 0.9927 F1-Score; IoT_CICIoMT attained the highest performance at 0.9952 accuracy and 0.9953 F1-Score. Across all datasets, balanced accuracy exceeded 0.99, MCC values ranged from 0.9795 to 0.9905, and ROC-AUC scores approached unity (≥ 0.9991). These results are particularly significant given that all three datasets employ TLS encryption, providing direct empirical support for the encryption-agnostic detection claim advanced in Section 5.

6.5.2. Combined multi-source training

To evaluate cross-environment scalability under rigorous statistical conditions, we aggregated samples from all three external datasets into a single corpus, stratified by both class label and source dataset, and applied 10-fold stratified cross-validation with results reported as mean $\pm$ standard deviation. This protocol directly tests whether a single model can generalize across heterogeneous IoT environments while providing the variance estimates necessary to rule out partition-dependent performance inflation.

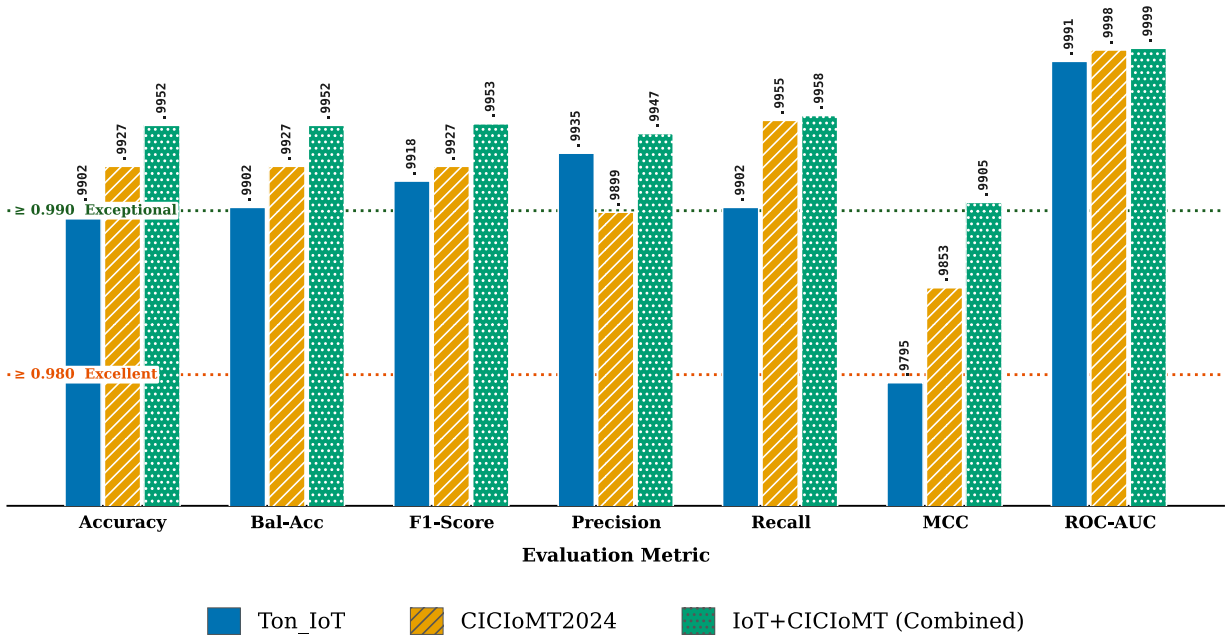


Fig. 6. Per-dataset validation results on three external TLS-encrypted IoT datasets.

Table 9

Combined multi-source training results under 10-fold stratified cross-validation. Results reported as mean $\pm$ standard deviation.

Model	Source Subset	B. Acc.	Precision	Recall	F1 Score	MCC	Train Time (s)	Inf. Time (s)
Logistic Reg.	OVERALL	0.9630 $\pm$ 0.0067	0.9631 $\pm$ 0.0055	0.9630 $\pm$ 0.0060	0.9630 $\pm$ 0.0059	0.9260 $\pm$ 0.0185	0.02 $\pm$ 0.0021	0.000282 $\pm$ 0.000017
	CICIoMT2024	0.9500 $\pm$ 0.0109	0.9545 $\pm$ 0.0098	0.9500 $\pm$ 0.0095	0.9499 $\pm$ 0.0107	0.9045 $\pm$ 0.0210	–	0.000094 $\pm$ 0.000009
	IoT_CICIoMT	0.9778 $\pm$ 0.0053	0.9787 $\pm$ 0.0052	0.9778 $\pm$ 0.0067	0.9778 $\pm$ 0.0063	0.9565 $\pm$ 0.0168	–	0.000094 $\pm$ 0.000010
	Ton_IoT	0.9611 $\pm$ 0.0096	0.9612 $\pm$ 0.0089	0.9611 $\pm$ 0.0092	0.9611 $\pm$ 0.0075	0.9223 $\pm$ 0.0210	–	0.000094 $\pm$ 0.000008
Random Forest	OVERALL	0.9815 $\pm$ 0.0041	0.9819 $\pm$ 0.0038	0.9815 $\pm$ 0.0039	0.9815 $\pm$ 0.0048	0.9634 $\pm$ 0.0111	0.02 $\pm$ 0.0029	0.013600 $\pm$ 0.001400
	CICIoMT2024	0.9944 $\pm$ 0.0038	0.9945 $\pm$ 0.0037	0.9944 $\pm$ 0.0032	0.9944 $\pm$ 0.0030	0.9889 $\pm$ 0.0074	–	0.004536 $\pm$ 0.000398
	IoT_CICIoMT	0.9667 $\pm$ 0.0088	0.9688 $\pm$ 0.0074	0.9667 $\pm$ 0.0072	0.9666 $\pm$ 0.0072	0.9354 $\pm$ 0.0207	–	0.004536 $\pm$ 0.000472
	Ton_IoT	0.9833 $\pm$ 0.0046	0.9839 $\pm$ 0.0044	0.9833 $\pm$ 0.0046	0.9833 $\pm$ 0.0054	0.9672 $\pm$ 0.0140	–	0.004536 $\pm$ 0.000578
LightGBM	OVERALL	0.9852 $\pm$ 0.0032	0.9853 $\pm$ 0.0040	0.9852 $\pm$ 0.0040	0.9852 $\pm$ 0.0038	0.9705 $\pm$ 0.0087	0.01 $\pm$ 0.0011	0.001156 $\pm$ 0.000082
	CICIoMT2024	0.9833 $\pm$ 0.0058	0.9834 $\pm$ 0.0058	0.9833 $\pm$ 0.0049	0.9833 $\pm$ 0.0054	0.9667 $\pm$ 0.0134	–	0.000385 $\pm$ 0.000026
	IoT_CICIoMT	0.9778 $\pm$ 0.0052	0.9787 $\pm$ 0.0067	0.9778 $\pm$ 0.0068	0.9778 $\pm$ 0.0067	0.9565 $\pm$ 0.0145	–	0.000385 $\pm$ 0.000035
	Ton_IoT	0.9944 $\pm$ 0.0038	0.9945 $\pm$ 0.0031	0.9944 $\pm$ 0.0034	0.9944 $\pm$ 0.0035	0.9889 $\pm$ 0.0072	–	0.000385 $\pm$ 0.000028

Table 9 presents results for three classifier architectures. Logistic Regression achieved an overall balanced accuracy of 0.9630 ± 0.0067 and F1-Score of 0.9630 ± 0.0059 , with per-dataset F1-Scores of 0.9499 ± 0.0107 (CICIoMT2024), 0.9778 ± 0.0063 (IoT_CICIoMT), and 0.9611 ± 0.0075 (Ton_IoT). Inference time remained sub-millisecond (0.000282 ± 0.000017 s), confirming deployment viability on resource-constrained edge devices. Random Forest and LightGBM yielded marginally higher overall F1-Scores (0.9815 ± 0.0048 and 0.9852 ± 0.0038 , respectively) at the cost of increased inference latency, reinforcing our thesis that lightweight classifiers paired with principled feature engineering achieve an optimal accuracy-efficiency trade-off. The narrow standard deviations across all three architectures confirm that the reported performance is stable across data partitions and not attributable to dataset-specific overfitting.

6.6. Outperforming existing methods in binary threat detection

To validate our findings, this section benchmarks our top-performing model from Case Study 1 against leading methods in the DoS/DDoS detection literature. As detailed in Table 10, the comparison reveals that our methodology does more than achieve superior accuracy; it fundamentally redefines the trade-off between generalizability, model complexity, and computational efficiency.

Our lightweight Logistic Regression model achieved an exceptional Macro F1-Score of 0.9794 on a challenging heterogeneous dataset of ten distinct DoS/DDoS attack vectors. This result surpasses the sophisticated GAN-AE model (0.976 F1-Score) [21] and substantially outperforms specialized systems on constrained threat scopes, including stacking ensembles (0.9537) [11], AdaBoost (0.9572) [16], and SVM (0.945) [22]. This performance advantage is visually demonstrated in Fig. 7, which contrasts our model

Table 10
Comparative performance analysis: proposed cross-layer model vs. existing DoS detection approaches.

Reference	Binary Task	Algorithm	Accuracy	F1-Score	Training Time (s)	Inference Time (s)	Key Features
our work	✓	Logistic Regression	0.9921	0.9794	0.0233	0.0007	Ultra-low latency with cross-layer validation
<i>Existing Literature Comparison</i>							
[8]	✓	Fuzzy Logic	0.9000	0.9090	—	—	Rule-based fuzzy inference system
[9]	✓	GRU	0.9608	0.9578	—	—	Multi-attack (DoS, MITM, Intrusion)
[21]	✓	GAN-AE	0.9760	0.9760	109.9540 ^a	0.1180 ^b	Generative adversarial approach
[11]	✓	Stacking Ensemble	0.9538	0.9537	250.0500	0.9400	Ensemble of multiple classifiers
[19]	✓	Decision Tree	0.9000	0.9130	9.1000	4.1800	Tree-based on packet-flow features
[17]	✓	k-NN	0.9900	0.9900	0.0130	0.1870	Distance-based classification
[16]	✓	AdaBoost	0.9500	0.9572	—	—	Boosting ensemble method
[18]	✓	MLP (Perceptron)	0.9150	0.9170	—	—	Low-rate DDoS detection
[22]	✓	SVM	0.9100	0.9450	—	—	High FAR ^c (24.4%) on IoTID20 dataset

Notes: ^a Training time per epoch on a custom dataset. ^b Inference time for a batch of 64 flows. ^c FAR: False Alarm Rate. Metrics marked with “—” were not reported in the original studies. For studies presenting multiple algorithms, the best-performing or most relevant model is shown.

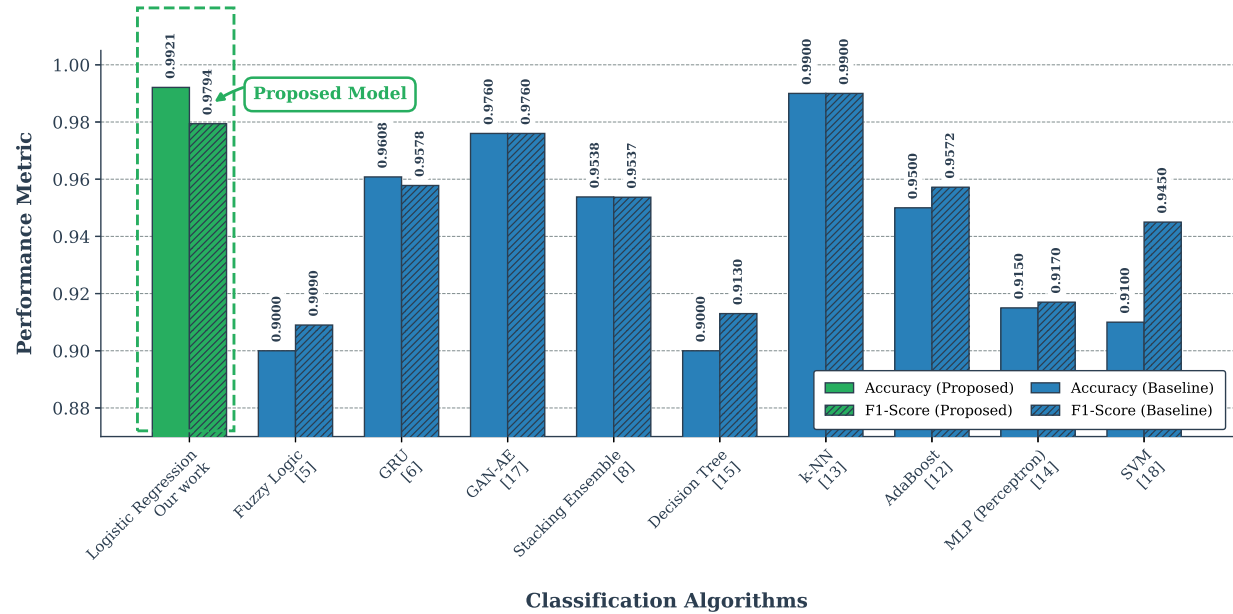


Fig. 7. F1-Score and accuracy comparison with state-of-the-art methods. Our proposed model (green) achieves the highest performance.

against these leading approaches across both F1-Score and Accuracy. This superiority extends to other metrics: while a hybrid CNN-LSTM [10] achieved 0.96 accuracy on a single vector, our model delivered a higher Balanced Accuracy of 0.9921 across a significantly broader threat landscape.

This work challenges the prevailing assumption that high model complexity is necessary for robust, multi-vector threat detection. The success of our lightweight Logistic Regression model contrasts sharply with the computationally intensive architectures, such as GANs [21], stacking ensembles [11], and hybrid CNN-LSTMs [9], which are often required for high performance on less diverse datasets. Our results demonstrate that by engineering strong and generalizable transport-layer features, the need for such intricate architectures is eliminated, even when addressing diverse attack vectors.

The primary advantage of our approach is its operational efficiency, which sets a new benchmark for the real-time deployment. Our Logistic Regression model achieves sub-millisecond inference (0.0007s) while classifying ten distinct attack types, a performance that is orders of magnitude beyond that of comparable studies. For example, our model is over 1300 times faster than the Stacking model (0.94s) [11] and over 4500 times faster than the Decision Tree (3.15s) [19]. Even compared to the highly optimized GAN-AE (0.0018s per flow) [21], our model’s single-instance prediction is faster and generalizes to over three times the number of attack vectors. This combination of accuracy, efficiency, and generalizability renders our methodology ideal for deployment in high-throughput gateways and resource-constrained IoT settings.



Fig. 8. Comparative performance analysis. (a) Baseline precision: per-class evaluation and (b) Baseline recall: per-class evaluation show the performance of the binary model when tested against individual attack classes. (c) F1-Score: Proposed vs. Baseline Model directly contrasts the harmonic mean of precision and recall, highlighting the significant performance improvement of the proposed multi-class method.

6.7. Comparing feature and sequence approaches in anomaly detection

To rigorously benchmark our proposed multi-class diagnostic framework, this section provides a detailed comparative analysis with the recent work of Swain et al. [12]. Although both studies targeted MQTT DoS/DDoS attacks, their methodologies diverged fundamentally. Swain et al. employed a binary anomaly classifier tested on isolated, per-attack scenarios, whereas our framework utilizes a unified multi-class model for simultaneous, fine-grained attack diagnostics. This distinction yields significant advantages in terms of diagnostic precision, performance consistency, and operational viability, as demonstrated in Fig. 8.

First, to ensure a methodologically sound comparison, we replicated the evaluation strategy of Swain et al. [12]. Specifically, our best-performing binary model from Case Study 1, Logistic Regression, was deployed and tested in a series of one-vs-all scenarios. The results, depicted as the “Baseline Method” in Fig. 8a and b, show a performance profile remarkably similar to that reported by Swain et al. [12]. Although the baseline model excels against volumetric attacks such as BF_DDoS and Delay_DDoS (Recall > 0.99), its efficacy deteriorates significantly for other classes. Most notably, the recall for Syn_TCP_DoS and Syn_TCP_DDoS attacks collapsed to a mere 0.065. This catastrophic failure mirrors the 0.065 recall reported by Swain et al. [12], confirming that simple binary or sequence-based models are fundamentally ill-equipped to detect this pervasive attack vector due to a shared critical vulnerability.

Next, we evaluated our proposed multi-class Artificial Neural Network (ANN) model, which is capable of processing all 11 classes simultaneously. Fig. 8 presents the performance of this model, labeled as the “Proposed Method”. The F1-Score comparison in Fig. 8c highlights a transformative improvement. The most striking result is the complete resolution of the baseline’s primary weakness: F1-Scores for Syn_TCP_DoS and Syn_TCP_DDoS catapult from 0.12 to 0.99 and 1.00, respectively, an improvement of over 87 percentage points. Significant gains were also evident for other previously challenging classes, such as Sub_DoS, where the F1-Score improved from 0.75 to 0.99 ($+0.240$).

Fig. 8d–f holistically summarize the impact of these gains. The radar chart (Fig. 8d) visually confirms superior performance across all attack categories, as the proposed method (orange area) consistently envelops the baseline (blue area). The category-

specific improvement chart (Fig. 8e) quantifies this, revealing that the Syn_TCP_DoS/DDoS category exhibited the greatest average improvement ($+0.615$). As the comprehensive summary shows (Fig. 8f), our model achieves an overall F1-Score improvement of $+0.213 \pm 0.339$ and attains near-perfect detection (≥ 0.99) for all 10 attack types. This statistically significant enhancement is further characterized by a medium effect size (Cohen's $d = 0.63$). The research impact is profound: reducing the F1-Score variance across classes by over 99% transformed an erratic, threat-dependent model into one that is consistently and robustly effective.

Finally, our framework delivers superior diagnostic capabilities without compromising operational feasibility. The inference time of the multi-class ANN model of only 0.1130 s is orders of magnitude faster than the 25 s latency reported by Swain et al. [12], thereby bridging the critical gap between high-fidelity classification and the low-latency requirements of real-time network security. Our multi-class approach provides a more granular diagnostic output while delivering superior accuracy, consistency, and response time well-suited for practical deployment.

6.8. Practical deployment scenarios

The framework's demonstrated accuracy and computational efficiency render it suitable for two primary deployment models. The first is as a lightweight intrusion detection service on edge devices, such as IoT gateways, where its sub-millisecond inference and minimal resource footprint facilitate real-time traffic inspection without imposing significant latency. A complementary approach involves its direct integration as a security module within an MQTT broker, enabling immediate, automated threat mitigation actions, including connection termination and client throttling. The empirical validation of these deployment models within a physical testbed represents the logical next phase of this research.

6.9. Limitations and future work

The proposed framework demonstrates efficacy, its primary limitations delineate clear avenues for future research. These include the current validation being confined to a single dataset, which warrants further investigation into model generalizability, and the potential scalability challenges of a centralized architecture in hyperscale IoT deployments. Accordingly, future research will proceed along three primary directions. First, we will investigate hybrid architectures that synergize the feature extraction capabilities of deep models with the computational efficiency of shallow classifiers to capture complex temporal patterns without compromising the real-time performance. Second, to mitigate the identified scalability and privacy constraints, we explore a federated learning framework, which enables the distributed training of a global detection model across multiple edge gateways without centralizing raw network traffic. Finally, the integration of Explainable AI techniques will be pursued to enhance model transparency, providing security analysts with crucial insights into the rationale behind threat classifications.

7. Conclusion

This research addresses the gap between detection accuracy and deployment feasibility in IoT intrusion detection. Unlike prior work limited to single attack vectors or binary classification, this study targets ten distinct DoS/DDoS attack types spanning both application (L7) and transport (L4) layers. Notably, the only prior attempt at cross-layer detection failed critically at Layer 4, achieving merely 0.065 Recall on TCP SYN floods. Our methodology overcomes this limitation through encryption-resilient feature engineering that operates on transport-layer metadata rather than payload inspection.

The proposed framework achieves an F1-Score of 0.9794 with sub-millisecond inference and a sub-kilobyte model footprint. Ablation analysis confirms that cross-layer feature integration is essential. Cross-dataset validation on three TLS-encrypted IoT datasets (Accuracy: 0.9902–0.9952) establishes generalization and validates the encryption-resilient design.

These findings demonstrate that principled feature engineering enables lightweight classifiers to succeed where complex architectures have failed, providing a deployment-ready framework for resource-constrained IoT environments.

CRediT authorship contribution statement

Hamza Allaga: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Resources, Software, Validation, Visualization, Writing – original draft, Writing – review & editing; **Abderrazak Farchane:** Supervision, Writing – review & editing; **Said Hakimi:** Supervision, Writing – review & editing; **Mohamed Biniz:** Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that there is no conflict of interest.

References

- [1] Statista Research Department, Internet of things (IoT) connected devices installed base worldwide from 2015 to 2025, 2016, <https://www.statista.com/statistics/471264/iot-number-of-connected-devices-worldwide/>.
- [2] S.N. Swamy, D.M. Anna, V. M. N., S.R. Kota, Enabling lightweight device authentication in message queuing telemetry transport protocol, IEEE Internet Things J. 11 (9) (2024) 15792–15807. <https://doi.org/10.1109/JIOT.2024.3349394>

- [3] G. Raja, S. Anbalagan, A. Ganapathisubramaniyan, M.S. Selvakumar, A.K. Bashir, S. Mumtaz, Efficient and secured swarm pattern multi-UAV communication, *IEEE Trans. Veh. Technol.* 70 (7) (2021) 7050–7058. <https://doi.org/10.1109/TVT.2021.3082308>
- [4] S. Rasool, A. Saleem, M. Iqbal, T. Dagiuklas, S. Mumtaz, Z. Ul Qayyum, DocsChain: blockchain based IoT solution for verification of degree documents, *IEEE Trans. Comput. Soc. Syst.* 7 (3) (2020) 827–837. <https://doi.org/10.1109/TCSS.2020.2973710>
- [5] M. Husnain, K. Hayat, E. Cambiaso, U.U. Fayyaz, M. Mongelli, H. Akram, S. Ghazanfar Abbas, G.A. Shah, Preventing MQTT vulnerabilities using IoT-enabled intrusion detection system, *Sensors* 22 (2) (2022) 567. <https://doi.org/10.3390/s22020567>
- [6] O. Yoachimik, J. Pacheco, Hyper-volumetric DDoS attacks skyrocket: Cloudflare's 2025 Q2 DDoS threat report, 2025, (The Cloudflare Blog). <https://blog.cloudflare.com/2025-q2-ddos-threat-report>
- [7] H. Allaga, M. Biniz, A. Farchane, MQTTEEB-D: a high-fidelity benchmark for real-time MQTT anomaly detection using machine learning techniques, *Ad Hoc Netw.* 181 (2026) 104062. <https://doi.org/10.1016/j.adhoc.2025.104062>
- [8] M.N. Hindia, A.W. Reza, A.K. Das, Secure-MQTT: an efficient fuzzy logic-based approach to detect DoS attack in MQTT protocol for internet of things, *EURASIP J. Wirel. Commun. Netw.* 2019 (1) (2019) 90.
- [9] H. Alaiz-Moretón, J. Avelaira-Mata, J.A. Ondicol-García, P. Tarrío, A.B. García, J.R. Villar, Multiclass classification procedure for detecting attacks on MQTT-IoT protocol, *Complexity* 2019 (2019) 6475736.
- [10] A. Alzahrani, T.H. Aldhyani, Artificial intelligence algorithms for detecting and classifying MQTT protocol internet of things attacks, *Electronics* 11 (22) (2022) 3837.
- [11] H. Zeghida, M. Boulaiche, R. Chikh, Securing MQTT protocol for IoT environment using IDS based on ensemble learning, *Int. J. Inf. Secur.* 22 (2023) 1075–1086.
- [12] M. Swain, N. Tripathi, K. Sethi, Identifying communication sequence anomalies to detect DoS attacks against MQTT, *Comput. Secur.* 157 (2025) 104526.
- [13] U. Tariq, I. Ahmed, A.K. Bashir, K. Shaukat, A critical cybersecurity analysis and future research directions for the internet of things: a comprehensive review, *Sensors* 23 (8) (2023) 4117. <https://doi.org/10.3390/s23084117>
- [14] Ahanger, U. I. A. S. A. T. U., Machine learning-inspired intrusion detection system for IoT: security issues and future challenges, *Comput. Electr. Eng.* 123 (2025), 110265.
- [15] I. Vaccari, G. Chiola, M. Aiello, M. Mongelli, E. Cambiaso, MQTTSet, a new dataset for machine learning techniques on MQTT, *Sensors* 20 (22) (2020) 6578.
- [16] S. Rachmadi, S. Mandala, D. Oktaria, Detection of DoS attack using AdaBoost algorithm on IoT system, in: 2021 International Conference on Data Science and Its Applications, IEEE, 2021, pp. 28–33.
- [17] H. Siddharthan, T. Deepa, P. Chandhar, SENMQTT-SET: an intelligent intrusion detection in IoT-MQTT networks using ensemble multi cascade features, *IEEE Access* 10 (2022) 33095–33110.
- [18] M. Al-Fayoumi, Q.A. Al-Haija, Capturing low-rate DDoS attack based on MQTT protocol in software defined-IoT environment, *Array* 19 (2023) 100316.
- [19] M.A. Khan, M.A. Khan, S.U. Jan, J. Ahmad, S.S. Jamal, A.A. Shah, N. Pitropakis, W.J. Buchanan, A deep learning-based intrusion detection system for MQTT enabled IoT, *Sensors* 21 (21) (2021) 7016.
- [20] P. Sitaraman, Enhancing DoS attack recovery in MQTT brokers for IoT systems through efficient scheduling algorithms, in: 2023 IEEE MIT Undergraduate Research Technology Conference, IEEE, 2023, pp. 1–4.
- [21] T.K. Boppana, P. Bagade, GAN-AE: an unsupervised intrusion detection system for MQTT networks, *Eng. Appl. Artif. Intell.* 119 (2023) 105805.
- [22] M. Ikhsanudin, V. Suryani, R.R. Pahlevi, DDoS attack detection on MQTT protocol using semi-supervised DBSCAN and support vector machine model, in: 2023 11th International Conference on Information and Communication Technology, IEEE, 2023, pp. 301–306.
- [23] A. Alatrani, L.F. Sikos, M. Johnstone, P. Szczyzyk, J.J. Kang, DoS/DDoS-MQTT-IoT: a dataset for evaluating intrusions in IoT networks using the MQTT protocol, *Comput. Netw.* 231 (2023) 109809.
- [24] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, A. Anber, TON_IoT telemetry dataset: a new generation dataset of IoT and IIoT for data-driven intrusion detection systems, *IEEE Access* 8 (2020) 165130–165150. <https://doi.org/10.1109/ACCESS.2020.3022862>
- [25] S. Dadkhah, E.C.P. Neto, R. Ferreira, R.C. Molokwu, S. Sadeghi, A.A. Ghorbani, CICIoMT2024: a benchmark dataset for multi-protocol security assessment in IoMT, *Internet Things* 28 (2024) 101351. <https://doi.org/10.1016/j.iot.2024.101351>
- [26] M. Rabbani, J. Gui, F. Nejati, Z. Zhou, A. Kaniyamattam, M. Mirani, G. Piya, I. Opushnyev, R. Lu, A.A. Ghorbani, Device identification and anomaly detection in IoT environments, *IEEE Internet Things J.* (2024). <https://doi.org/10.1109/JIOT.2024.3512345>
- [27] N. Moustafa, J. Slay, UNSW-NB15: a comprehensive data set for network intrusion detection systems, in: 2015 Military Communications and Information Systems Conference, 2015.
- [28] C.I.f. Cybersecurity, CIC-IDS2017 dataset, 2017, (<https://www.unb.ca/cic/datasets/ids-2017.html>).
- [29] M.I.o. Technology, 1999 DARPA intrusion detection evaluation dataset, 1999.
- [30] N. Koroniots, N. Moustafa, E. Sitnikova, B. Turnbull, Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-IoT dataset, *Future Gener. Comput. Syst.* 100 (2019) 779–796.
- [31] CAIDA, The CAIDA “DDoS attack2007” dataset, 2007.
- [32] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, 2018, pp. 108–116.
- [33] M.A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, H. Janicke, Edge-IoTset: a new comprehensive realistic cyber security dataset of IoT and IIoT applications: Centralized and federated learning, 2022. <https://doi.org/10.21227/mbc1-1h68>
- [34] I. University of California, KDD cup 1999 data, 1999.
- [35] A. Aqachtoul, K. Karam, A. Elamrani, M. Najib, N. Rafalia, M. Bakhouya, MQTTEEB-D: a real-world IoT cybersecurity dataset for AI-powered threat detection in MQTT networks, 2025. <https://doi.org/10.17632/jftfn6tr.1>
- [36] H. Hindy, E. Bayne, M. Bures, R. Atkinson, C. Tachtatzis, X. Bellekens, Machine learning based IoT intrusion detection system: an MQTT case study, in: *International Networking Conference*, 2020.
- [37] Y. Meidan, N-BaloT, 2025. <https://doi.org/10.21227/y9de-qj71>
- [38] Custom dataset: flow-based MQTT attack dataset, 2022, <https://anonymous.4open.science/r/Flow-Based-MQTT-AttackData-Set-E768/>.
- [39] E.C.P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A.A. Ghorbani, CICIoT2023: a real-time dataset and benchmark for large-scale attacks in IoT environment, *Sensors* 23 (13) (2023) 5941. <https://doi.org/10.3390/s23135941>
- [40] M. Rabbani, J. Gui, F. Nejati, Z. Zhou, A. Kaniyamattam, M. Mirani, G. Piya, I. Opushnyev, R. Lu, A.A. Ghorbani, Device identification and anomaly detection in IoT environments, *IEEE Internet Things J.* (2024). Advance online publication. <https://doi.org/10.1109/JIOT.2024.3522863>